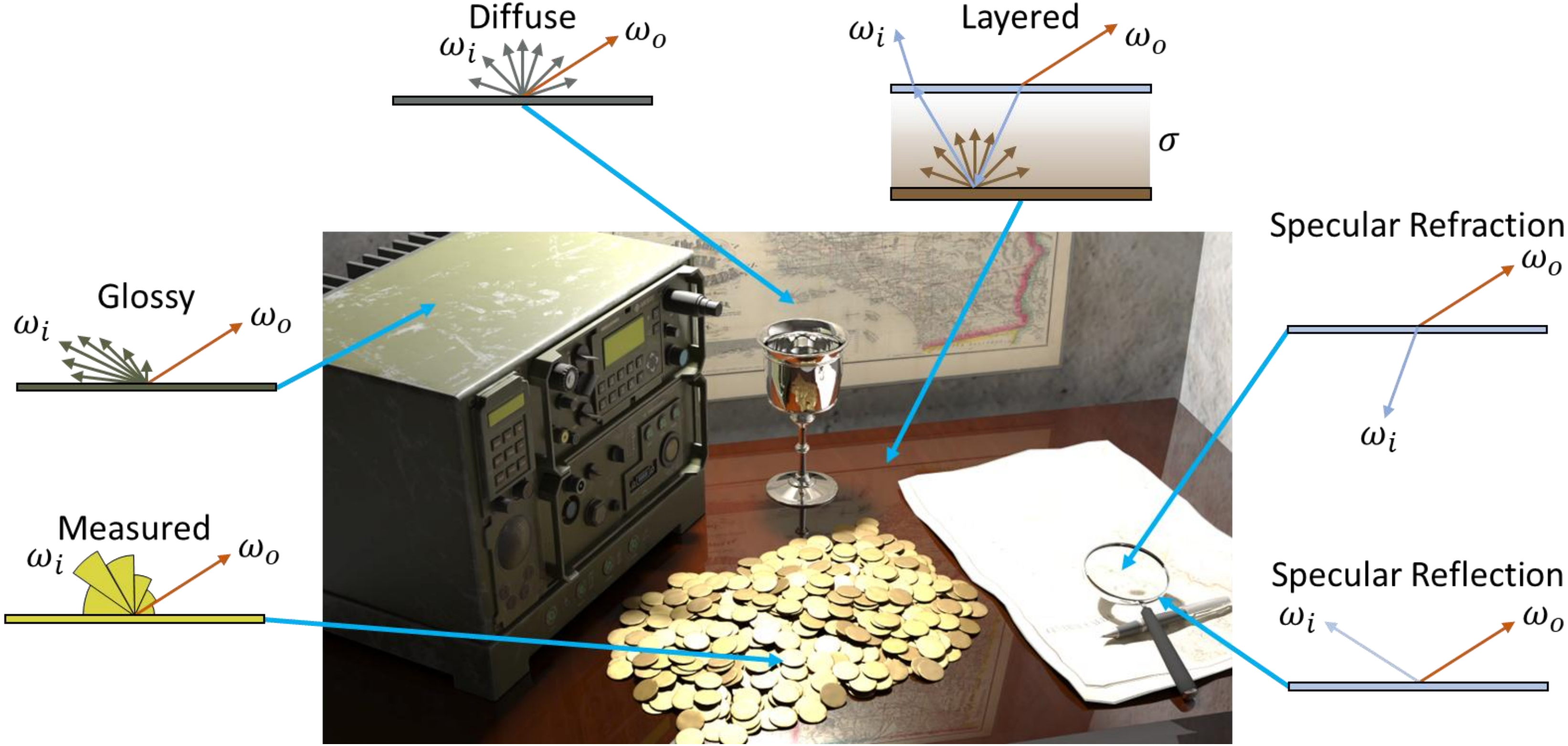
# Advanced Computer Graphics

*Materials*

Dr. Tom Bashford-Rogers

# Materials Overview



# Materials Interface

- Three pieces of core functionality
  - Evaluation
  - Sampling
  - Returning the PDF of sampling

# Materials Interface

- Three pieces of core functionality
  - Example

```
Vec3 sample(const ShadingData& shadingData, Sampler* sampler, Colour& reflectedColour, float& pdf)
Colour evaluate(const ShadingData& shadingData, const Vec3& wi)
float PDF(const ShadingData& shadingData, const Vec3& wi)
```

  - Sample returns sampled direction, reflected colour and PDF (shared computation)

# Shading Basis

- Usually transform from world space to convenient basis
- Usually shading normal $n$ to Z Up
- Use Frame class from CG: $R_n$
  - Build from Gram-Schmidt
    - Transform from local to world: $\omega_{world} = R_n \, \omega_{local}$
    - Transform from world to local: $\omega_{local} = R_n^{-1} \, \omega_{world}$

# Shading Basis

- Evaluation
  - Transform to local z-up space

```
Vec3 wo = shadingData.frame.toLocal(shadingData.wo);
```

- Sampling
  - Sample in z-up space
  - Transform to world

```
wi = shadingData.frame.toWorld(wi);
```

- PDF
  - Transform from world to local z-up

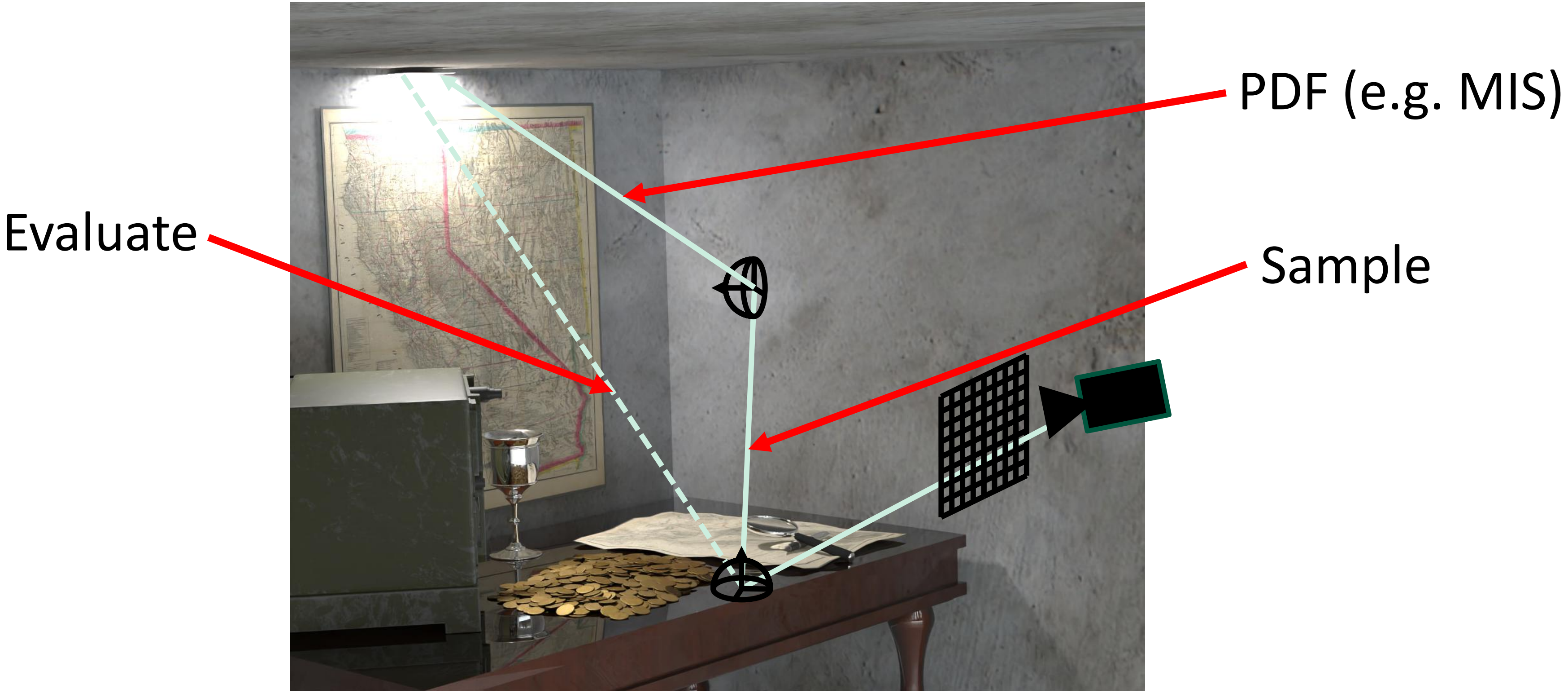
# Integration into Renderers

- Replace cosine sampling in renderer with

```
Colour indirect;
float pdf;
Vec3 wi = shadingData.bsdf->sample(shadingData, sampler, indirect, pdf);
```

- Now will sample proportional to BSDF (once implemented)
- Will need PDF for MIS

# Materials Interface

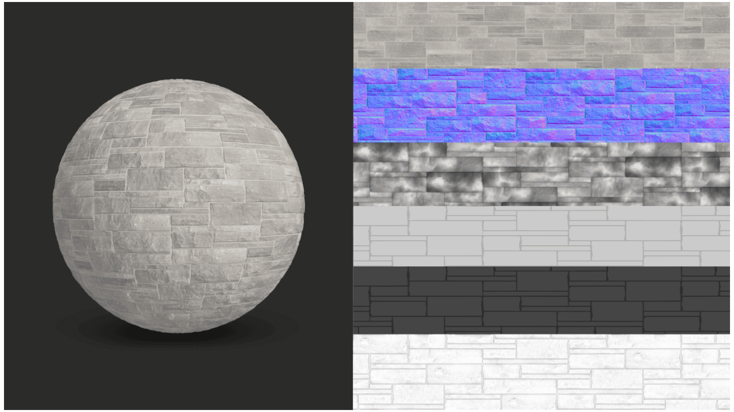


# Implementation Details

- Materials.h contains BSDFs we will use
- Have placeholder sampling, evaluation and PDF
- You will need to replace these

# Common Attributes

- Colour at surface (albedo) $\rho(x)$
- Other attributes (e.g. roughness)
  - Often varies across surface
  - Stored in a texture map



# BSDF Recap

- How much radiance is reflected in direction $\omega_o$ per unit of incident energy (irradiance) arriving from direction $\omega_i$

$$f_r(x, \omega_i, \omega_o) = \frac{dL_o(x, \omega_o)}{L_i(x, \omega_i)\cos(\theta_i) d\omega_i}$$

- Always positive $f_r(x, \omega_i, \omega_o) \geq 0$

# Light Transport

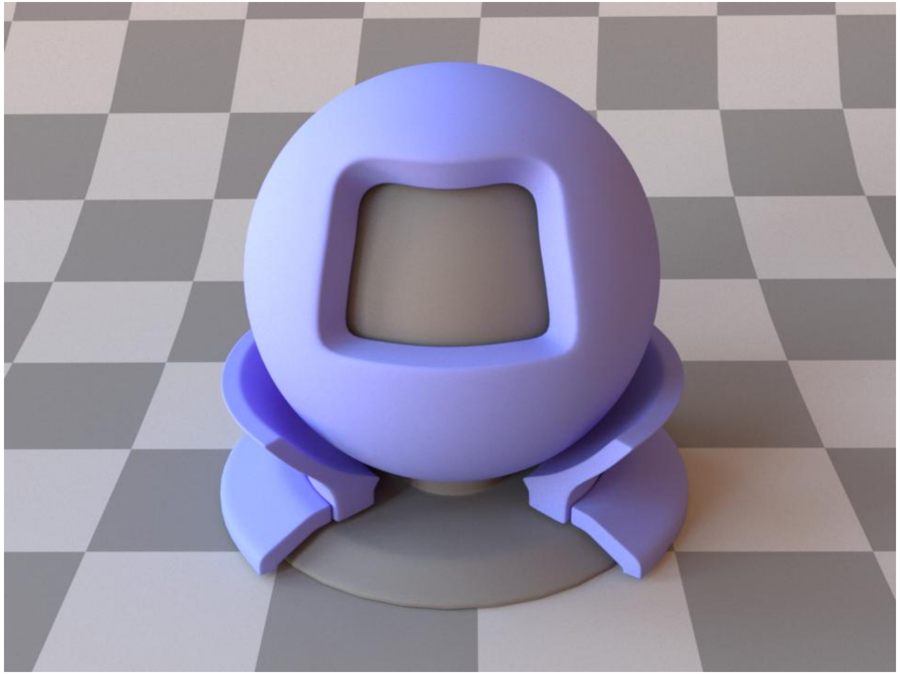
- Materials: BRDF Properties
  - Units $sr^{-1}$
  - Helmholtz Reciprocity

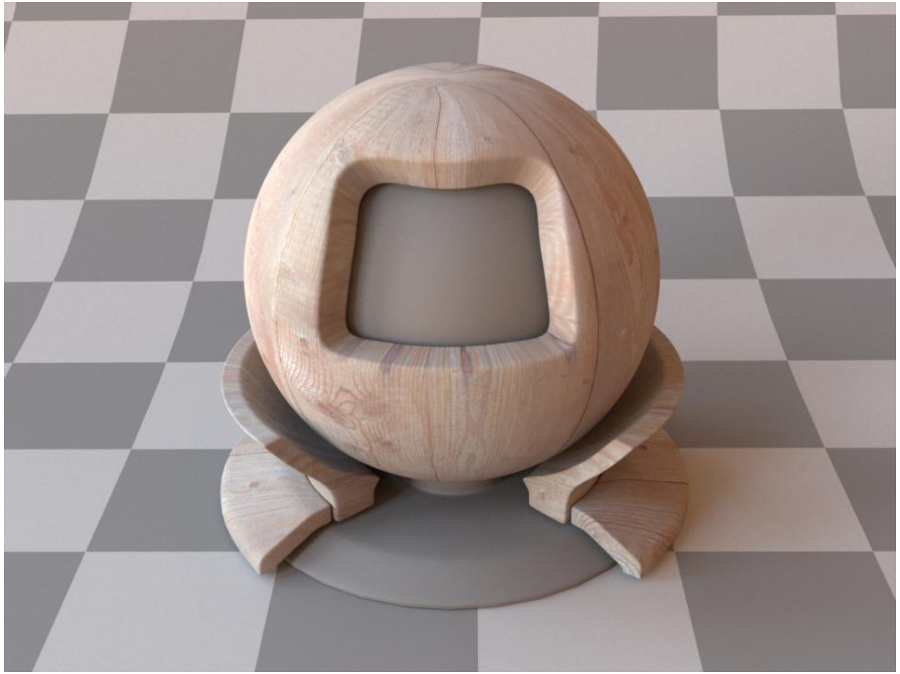
$$f_r(x, \omega_i, \omega_o) = f_r(x, \omega_o, \omega_i)$$

  - Energy Conservation

$$\int_\Omega f_r(x, \omega_i, \omega_o)\, \cos\theta\, d\omega_i \leq 1$$

# Diffuse



*Images from https://mitsuba.readthedocs.io/en/stable/src/generated/plugins_bsdfs.html*

# Diffuse

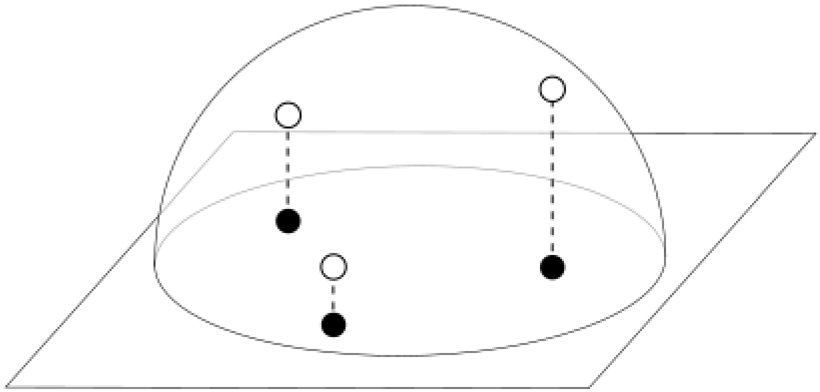

- Reflects light equally in all directions
- Want to reflect desired colour: $\rho(x)$
- Cannot simply use $\rho(x)$
  - BSDF needs to integrate to <= 1 over hemisphere
  - Need to normalize

# Diffuse


- Know that $\rho(x) = \int_\Omega f_r(x, \omega_i, \omega_o)(n \cdot \omega_i) d\omega_i$
- And diffuse BSDF is constant for all directions $fr(x, \omega_i, \omega_o) = C$
- So $\rho(x) = C \int_\Omega (n \cdot \omega_i) d\omega_i = C\pi$
- Rearranging and substituting back the BSDF $fr(x, \omega_i, \omega_o) = \frac{\rho(x)}{\pi}$

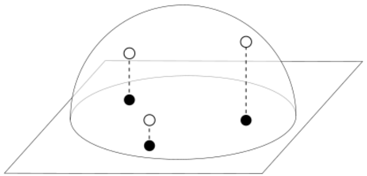
# Diffuse


- Sampling and PDF
  - Constant BSDF
  - But can sample cosine term
  - Cosine hemisphere PDF



# Diffuse


- Functionality
  - Evaluation: $fr(x, \omega_i, \omega_o) = \frac{\rho(x)}{\pi}$
  - Sampling: $\theta = \cos^{-1}(\sqrt{\xi_1})$
    $\phi = 2\pi\xi_2$



  - PDF: $p(\omega) = \frac{\cos(\theta)}{\pi}$

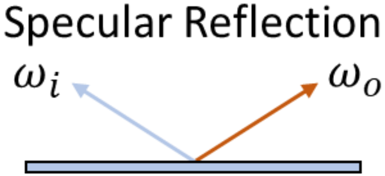
# Diffuse

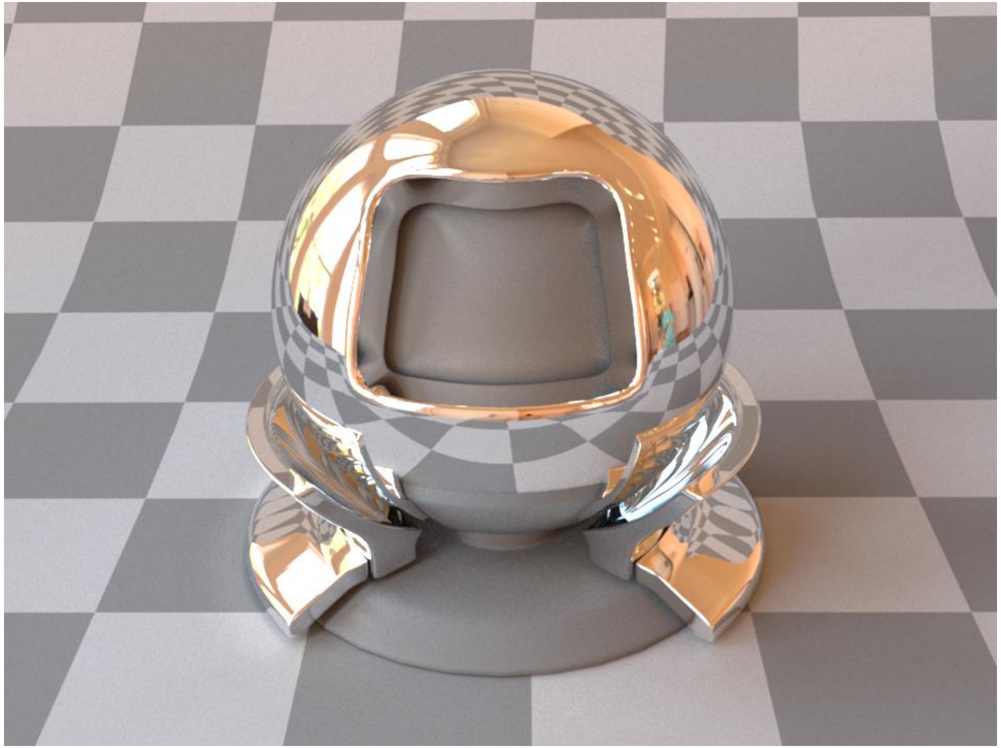
- Implementation
  - Sampling
    - Add cosine sampling code to

```
Vec3 sample(const ShadingData& shadingData, Sampler* sampler, Colour& reflectedColour, float& pdf)
```

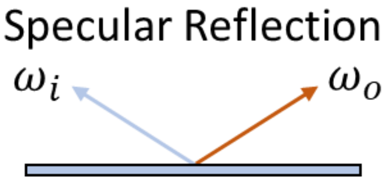
  - PDF
    - Convert to local frame
    - Evaluate cosine PDF `SamplingDistributions::cosineHemispherePDF`

# Mirror



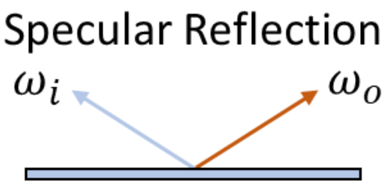



# Mirror




- Perfect (specular) reflection
  - In world space: $\omega_r = \omega - 2(\omega \cdot n)\, n$
  - In local frame: $\omega_r = (-\omega_x, -\omega_y, \omega_z)$
- BSDF derivation:
  - Need reflectance only in direction $\omega_r$ where $\omega = \omega_o$
  - Want $L_o(x, \omega_o) = L_i(x, \omega_r)$, will generalize later

# Mirror

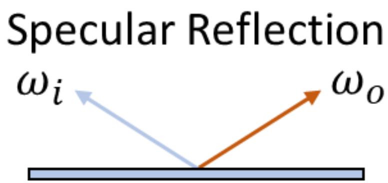



- BSDF derivation:
  - Can use Dirac Delta distribution

$$\int_{-\infty}^{\infty} \delta(x) f(x)\, dx = f(0)$$

$$\int_{-\infty}^{\infty} \delta(x - x_0) f(x)\, dx = f(x_0)$$

$$\int_{-\infty}^{\infty} \delta(x)\, dx = 1$$




# Mirror

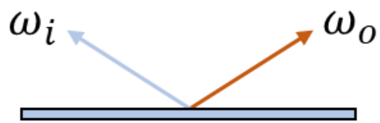
- BSDF derivation:
  - Can try $fr(x, \omega_i, \omega_o) = \delta(\omega_i - \omega_r)$
  - However, if we try to integrate

$$L_o(x, \omega_o) = \int_\Omega \delta(\omega_i - \omega_r) L_i(x, \omega_i)(\omega_i \cdot n) d\omega_i = L_i(x, \omega_r)(\omega_r \cdot n)$$

  - Extra factor of $(\omega_r \cdot n)$
  - Solution: $fr(x, \omega_i, \omega_o) = \frac{\delta(\omega_i - \omega_r)}{(\omega_r \cdot n)}$

# Mirror

Specular Reflection




- **Functionality**
  - Evaluation: $fr(x, \omega_i, \omega_o) = \frac{\delta(\omega_i - \omega_r)}{(\omega_r \cdot n)}$
  - Sampling: $\omega_r = (-\omega_x, -\omega_y, \omega_z)$
  - PDF: $p(\omega) = 0$
    - Why?

# Mirror

- Implementation
  - Sampling
    - Convert `shadingData.wo` to local space
    - Reflect x and y
    - Convert back to world space
    - `reflectedColour` is `albedo->sample(shadingData.tu, shadingData.tv)`
      - Divide by $(\omega_r \cdot n) = \cos(\theta)$

# Reflection and Transmission

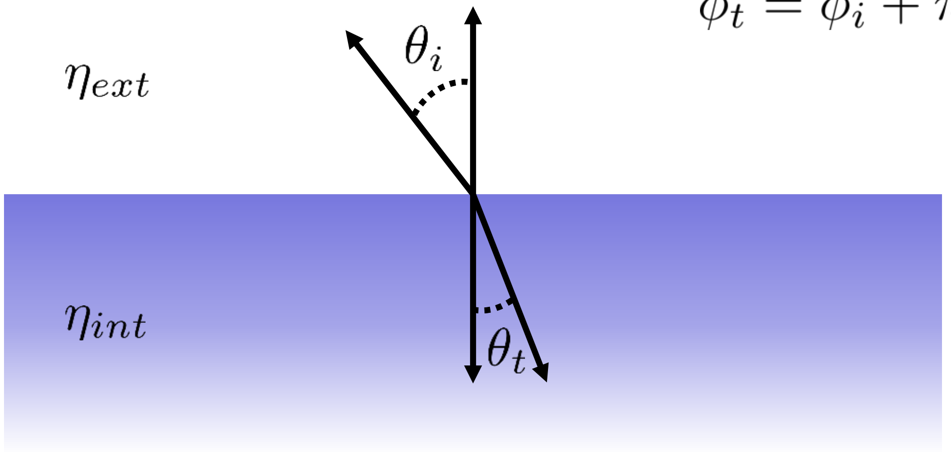
- So far we have looked at extremes
  - Not really physically realistic
- Need more principled approach
- Start with reflection and transmission / refraction
  - Already covered perfect reflection

# Transmission

- Specular Transmission
- Based on Snell's Law

$$\eta_{ext} \sin(\theta_i) = \eta_{int} \sin(\theta_t)$$

$$\phi_t = \phi_i + \pi$$



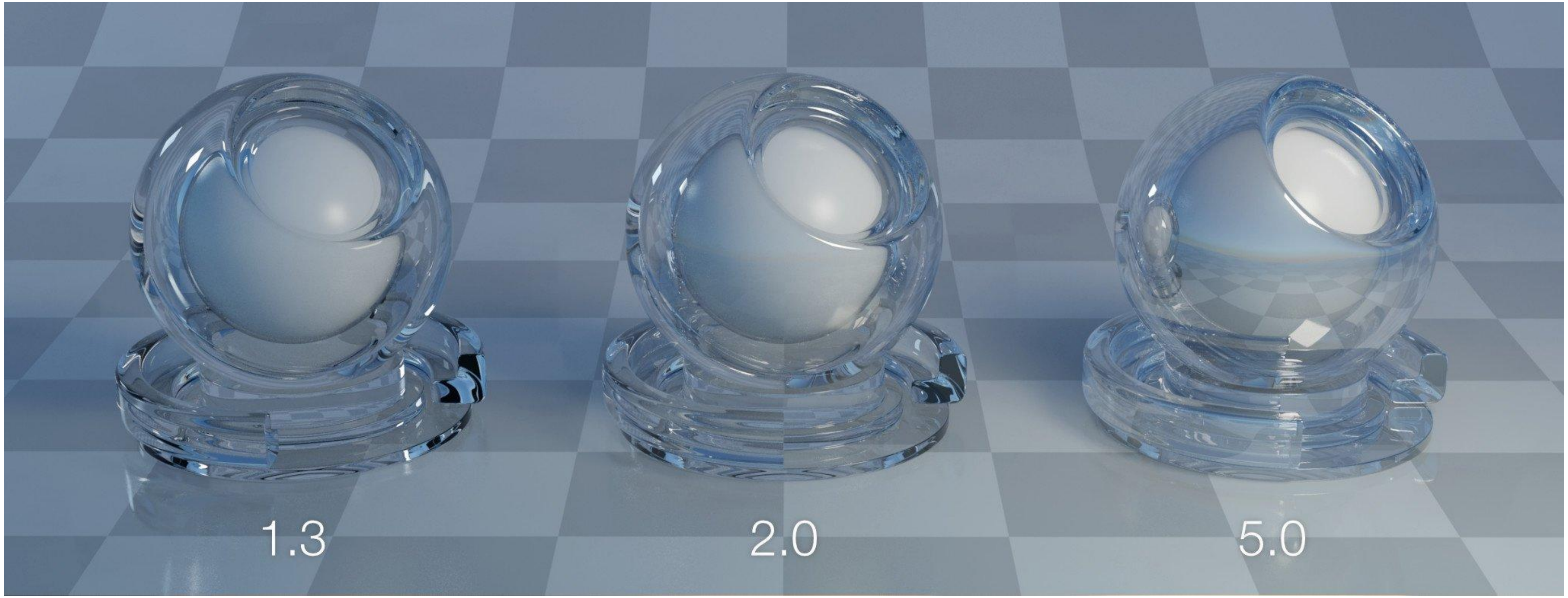
# Transmission

- Index of Refraction: $\eta$

| Material | Index of Refraction |
| --- | --- |
| Air | ~1 |
| Water | ~1.33 |
| Fused Silica (Quartz) | ~1.46 |
| Crown Glass | ~1.52 |
| Flint Glass | ~1.62 |
| Diamond | ~2.42 |

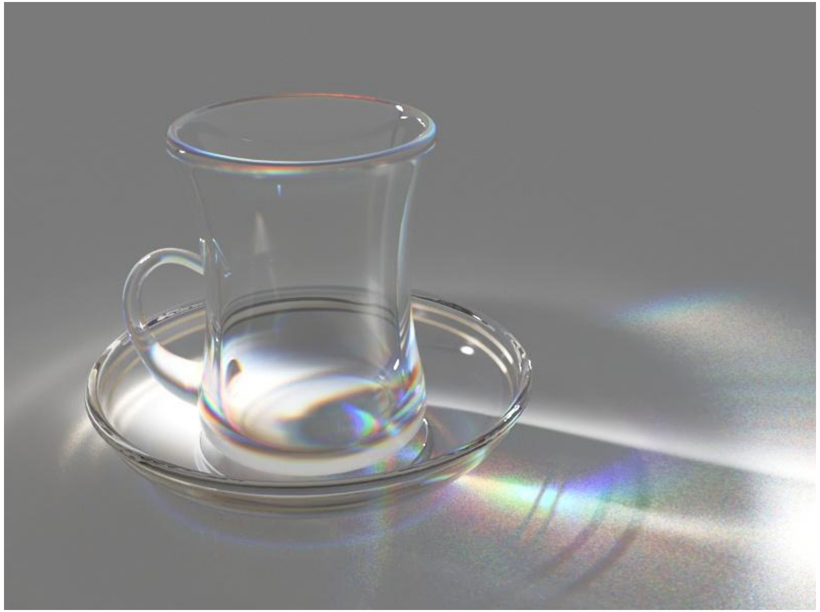
# Transmission

- Index of Refraction: $\eta$



# Transmission

- Index of Refraction: $\eta$
  - Should vary with wavelength (can ignore if not spectral)
  - Dispersion



# Transmission

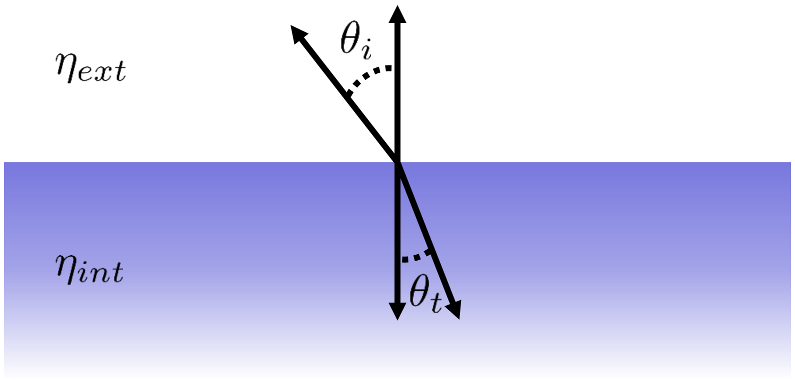
- How to compute transmission direction $\omega_t$
  - Rewrite Snell's Law: $\sin(\theta_t) = \eta \sin(\theta_i)$

$$\eta = \frac{\eta_{ext}}{\eta_{int}}$$

  - Calculate $\theta_t$ : $\cos(\theta_t) = \sqrt{1 - \eta^2 \sin^2(\theta_i)}$
    - From trig identities
    - $\sin^2(\theta_i) = 1 - \cos^2(\theta_i) = 1 - \omega_{i,z} \cdot \omega_{i,z}$

# Transmission

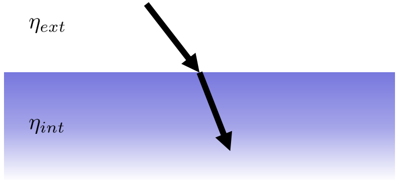
- How to compute transmission direction $\omega_t$
  - Therefore $\omega_t = (\sin(\theta_t)\cos(\phi_t), \sin(\theta_t)\sin(\phi_t), -\cos(\theta_t))$
  - Or $\omega_t = (-\eta\,\omega_{i,x}, -\eta\,\omega_{i,y}, -\cos(\theta_t))$

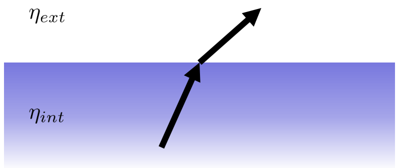


# Transmission

- How to compute transmission direction $\omega_t$
  - Direction: $\omega_t = (\,-\eta\,\omega_{i,x},\,-\eta\,\omega_{i,y},\,-\cos(\theta_t)\,)$
- In practice
  - $\eta$ depends on direction
    - If entering $\eta = \dfrac{\eta_{ext}}{\eta_{int}}$
    - If exiting $\eta = \dfrac{\eta_{int}}{\eta_{ext}}$

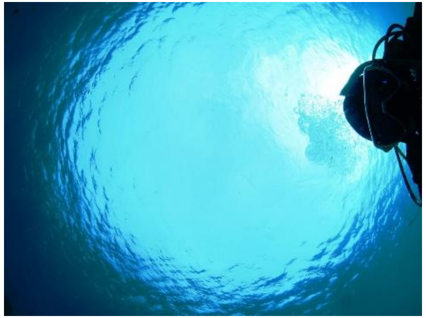
$$\omega_t = (\,-\eta\,\omega_{i,x},\,-\eta\,\omega_{i,y},\,\cos(\theta_t)\,)$$

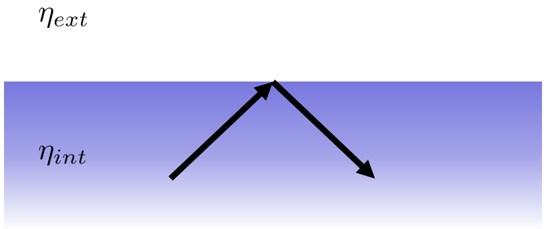


# Transmission

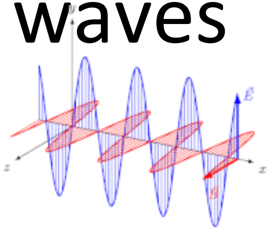
- Specular Transmission
  - What if in $\cos(\theta_t) = \sqrt{1 - \eta^2 \sin^2(\theta_i)}$
    - $\eta^2 \sin^2(\theta_i) > 1$?
  - Total Internal Reflection
    - Critical Angle
  - Return reflection vector

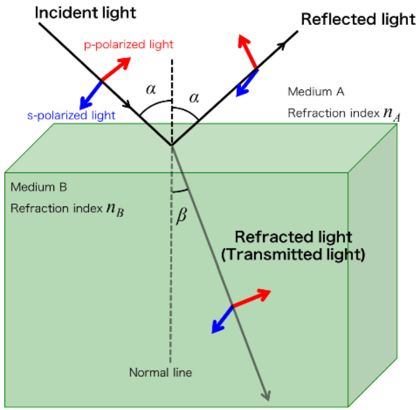




# Fresnel

- How much light is reflected or transmitted into a surface
  - We know how to compute these directions
- Given by Fresnel Equations
- Based on polarization of waves





# Fresnel

- Calculates amplitude of reflected wave given input wave
  - Polarization needs to be considered
- Expressed as a ratio
  - Different for Dielectrics and Conductors

# Fresnel

- Dielectric
  - Average squared parallel and perpendicular ratios
  - Parallel: $$F^{||}(\omega) = \frac{A_r^{||}}{A_r^{||}} = \frac{\cos(\theta_i) - \eta\cos(\theta_t)}{\cos(\theta_i) + \eta\cos(\theta_t)}$$
  - Perpendicular $$F^{\perp}(\omega) = \frac{A_r^{\perp}}{A_r^{\perp}} = \frac{\eta\cos(\theta_i) - \cos(\theta_t)}{\eta\cos(\theta_i) + \cos(\theta_t)}$$
  - Average $$F(\omega) = \frac{F^{||}(\omega)^2 + F^{\perp}(\omega)^2}{2}$$

# Fresnel

- Dielectric Implementation
  - Add implementation in `ShadingHelper` class
  - Fill in

```
static float fresnelDielectric(float cosTheta, float iorInt, float iorExt)
```

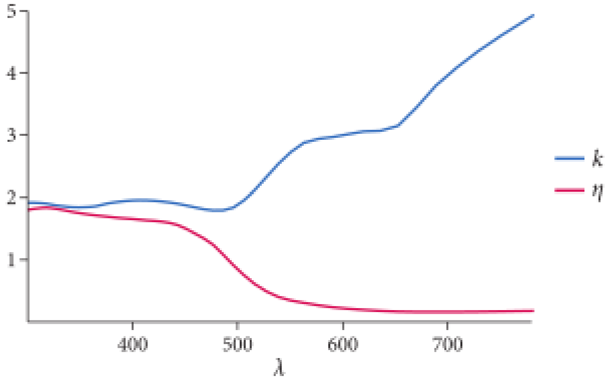
  - Need to implement specular transmission (refraction)
  - Assignment

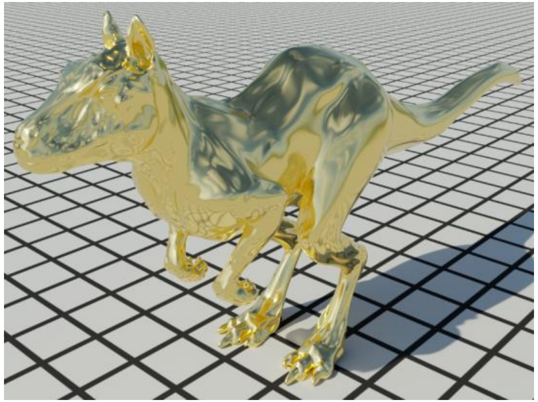
# Fresnel

- Conductor
  - IOR is complex (from wave description)
    - $\eta_t$ component as before
    - $k$ describes the exponential attenuation of light in conductors

# Fresnel

- Conductor
  - Highly wavelength dependent
  - Can convert to RGB



# Fresnel

- Conductor (returns colour)
  - Average squared parallel and perpendicular ratios
  - Parallel:

$$F^{||}(\omega) = \frac{(\eta + i\,k)^2\,\cos\theta_i - \sqrt{(\eta + i\,k)^2 - \sin^2\theta_i}}{(\eta + i\,k)^2\,\cos\theta_i + \sqrt{(\eta + i\,k)^2 - \sin^2\theta_i}}$$

  - Perpendicular:

$$F^{\perp}(\omega) = \frac{\cos\theta_i - \sqrt{(\eta + i\,k)^2 - \sin^2\theta_i}}{\cos\theta_i + \sqrt{(\eta + i\,k)^2 - \sin^2\theta_i}}$$

# Fresnel

- Conductor
  - But only need real components (note computes squared values
  - Parallel: $$F^{||}(\omega)^2 = \frac{(\eta^2 + k^2)\cos^2\theta_i - 2\,\eta\cos\theta_i + \sin^2\theta_i}{(\eta^2 + k^2)\cos^2\theta_i + 2\,\eta\cos\theta_i + \sin^2\theta_i}$$
  - Perpendicular: $$F^{\perp}(\omega)^2 = \frac{(\eta^2 + k^2 - 2\,\eta\cos\theta_i + \cos^2\theta_i)}{(\eta^2 + k^2 + 2\,\eta\cos\theta_i + \cos^2\theta_i)}$$
  - Average as before $$F(\omega) = \frac{F^{||}(\omega)^2 + F^{\perp}(\omega)^2}{2}$$

# Fresnel

- Conductor Implementation
  - Add implementation in `ShadingHelper` class
  - Fill in

```
static Colour fresnelConductor(float cosTheta, Colour ior, Colour k)
```

  - Assignment

# Fresnel

- Schlick's Approximation
  - Fast approximation used in games

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5$$

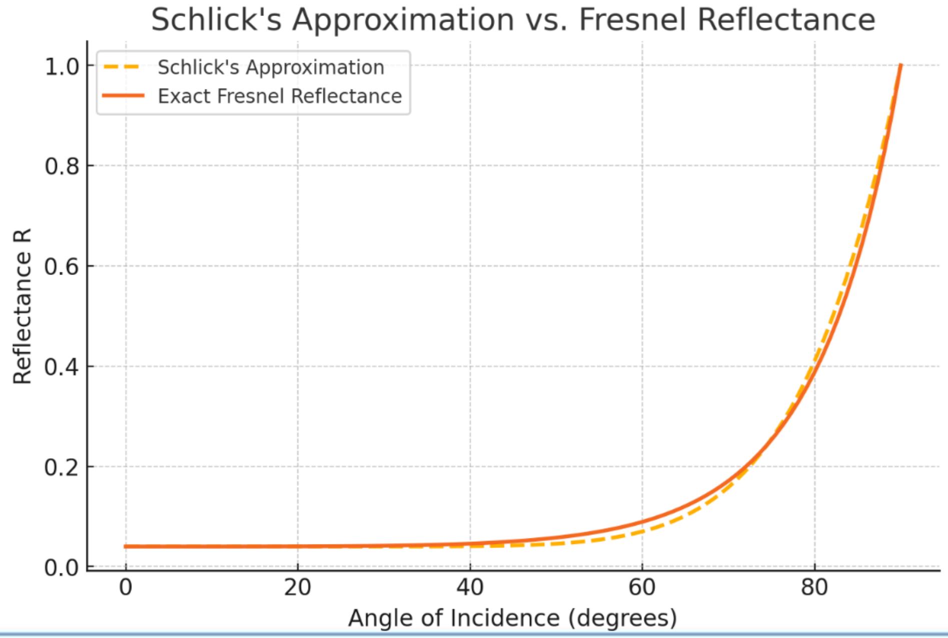
  - Where $F_0$ is reflectance at normal incidence

$$F_0 = \left(\frac{\eta_i - \eta_t}{\eta_i + \eta_t}\right)^2$$

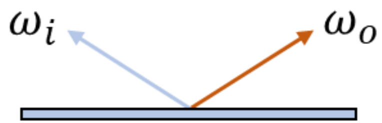
  - Based on rational approximations

# Fresnel

- Schlick's Approximation



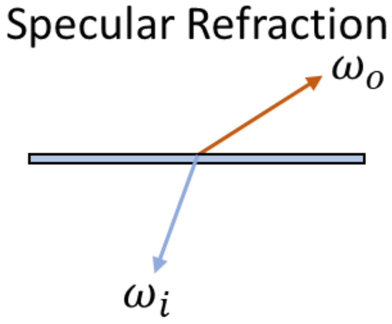
Specular Reflection




# Improved Mirror

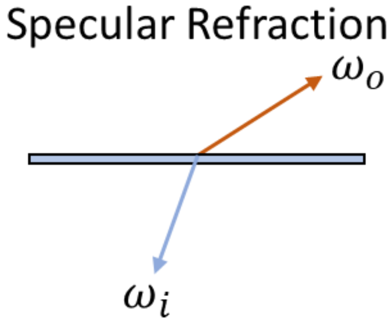
- Functionality
  - Evaluation: $fr(x, \omega_i, \omega_o) = F(\omega_r)\frac{\delta(\omega_i - \omega_r)}{(\omega_r \cdot n)}$
  - Sampling: $\omega_r = (-\omega_x, -\omega_y, \omega_z)$
  - PDF: $p(\omega) = 0$

# Glass

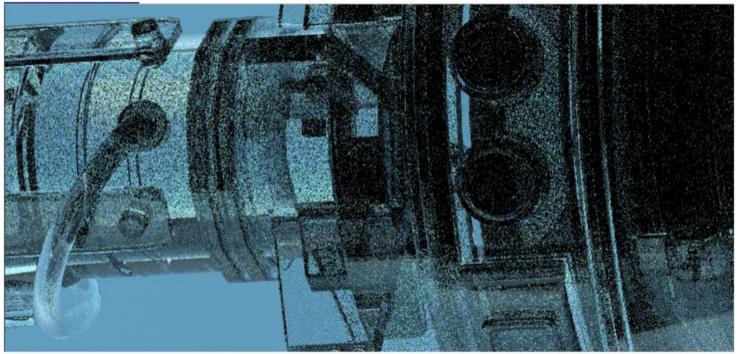


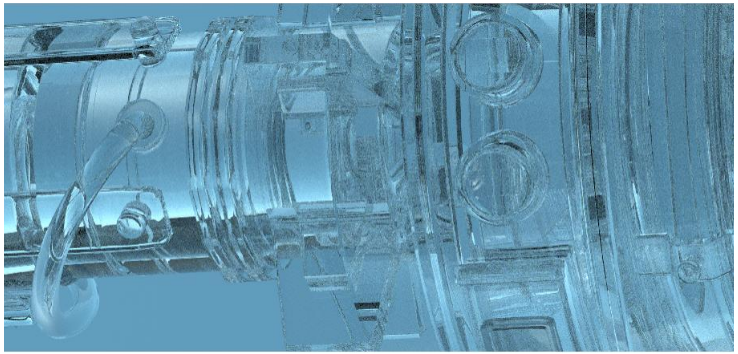

- Similar to mirror but can refract
  - Reflection: $\omega_r$
  - Refraction: $\omega_t$
- Only one event can happen, weighted by Fresnel

# Glass

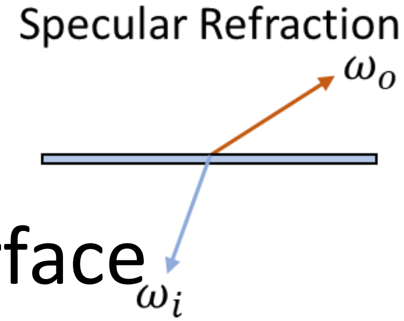



- Sampling
  – Need to pick reflection or transmission
  – Can pick component proportional to Fresnel





# Glass

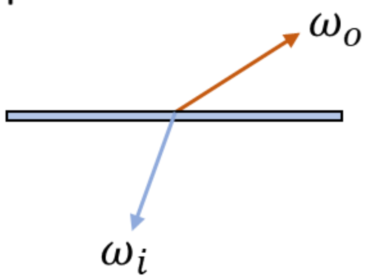



- Radiance is scaled through an interface
  - Accounts for energy preservation

$$L_i = \frac{\eta_t^2}{\eta_i^2} L_o$$

  - So $fr(x, \omega_i, \omega_o) = F(\omega_r)\frac{\delta(\omega_i - \omega_r)}{|\omega_r \cdot n|} + \frac{\eta_t^2}{\eta_i^2}(1 - F(\omega_r))\frac{\delta(\omega_i - \omega_t)}{|\omega_t \cdot n|}$

# Glass

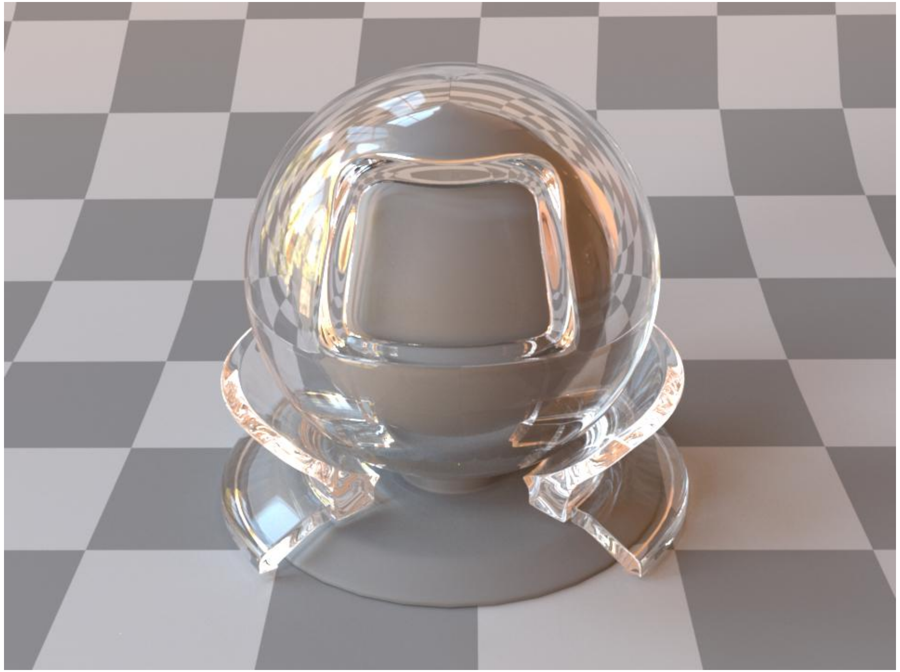


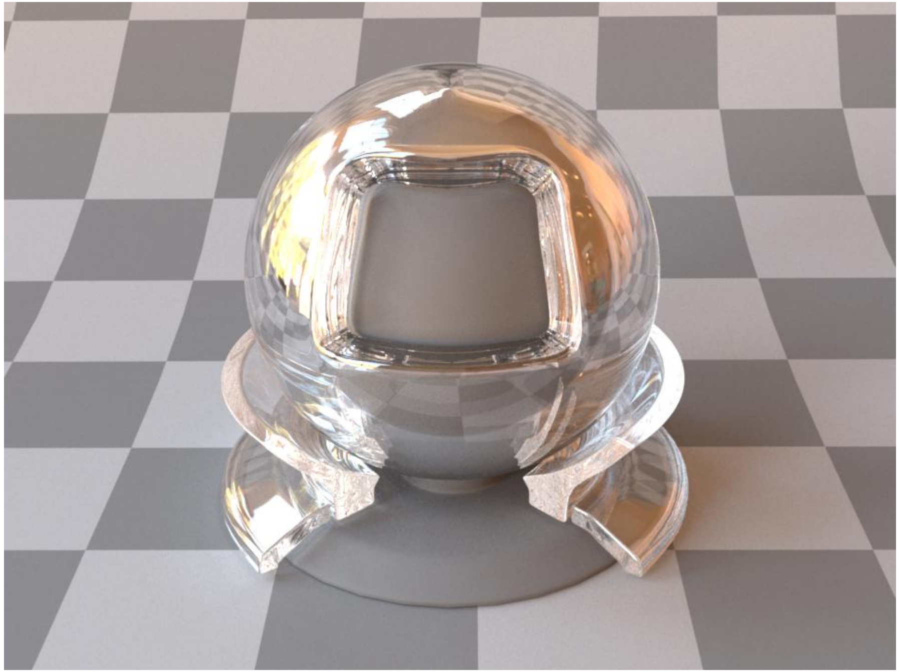

- Functionality
  - Evaluation: $fr(x,\omega_i,\omega_o) = F(\omega_r)\frac{\delta(\omega_i-\omega_r)}{|\omega_r\cdot n|} + \frac{\eta_t^2}{\eta_i^2}(1-F(\omega_r))\frac{\delta(\omega_i-\omega_t)}{|\omega_t\cdot n|}$
  - Sampling:
    - Reflection $\omega_r = (-\omega_x, -\omega_y, \omega_z)$
    - Transmission $\omega_t = (\sin(\theta_t)\cos(\phi_t), \sin(\theta_t)\sin(\phi_t), -\cos(\theta_t))$
  - PDF: $p(\omega) = 0$

# Glass

- Implementation
  - Assignment!

# Specular Surfaces





# Glossy: Phong


- Oldest glossy model
- Good for plastic materials
- Used to be used in games

# Glossy: Phong


- Glossy part defined as a lobe with an exponent $e$

$$fr(x, \omega_i, \omega_o) = \frac{e+2}{2\pi} \max(0, \omega_r \cdot \omega_i)^e$$

- Usually written as weighted sum of diffuse and glossy

$$fr(x, \omega_i, \omega_o) = k_d \frac{\rho(x)}{\pi} + k_s \frac{e+2}{2\pi} \max(0, \omega_r \cdot \omega_i)^e$$

# Glossy: Phong


- Sampling:
  - Cannot sample directly
  - But can sample z-up lobe and rotate
    - Sample lobe $$\theta_{lobe} = \cos^{-1}(\xi_1^{\frac{1}{e+1}})$$ $$\phi_{lobe} = 2\pi\xi_2$$

$$\omega_{lobe} = (\sin(\theta_{lobe})\cos(\phi_{lobe}), \sin(\theta_{lobe})\sin(\phi_{lobe}), \cos(\theta_{lobe}))$$

  - Create frame aligned along $\omega_r$ : $R_{\omega_r}$
  - Rotate vector: $\omega_i = R_{\omega_r}\omega_{lobe}$

# Glossy: Phong


- PDF: $p(\omega) \propto \max(0, \omega_r \cdot \omega)^e$
  - Normalize $$\int_\Omega \max(0,\ \omega_r \cdot \omega)^n \, \mathrm{d}\omega = \frac{2\pi}{n+1}$$
  - So $$p(\omega) = \tfrac{e+1}{2\pi} \max(0, \omega_r \cdot \omega)^e$$
- Integrate for CDF

# Glossy: Phong

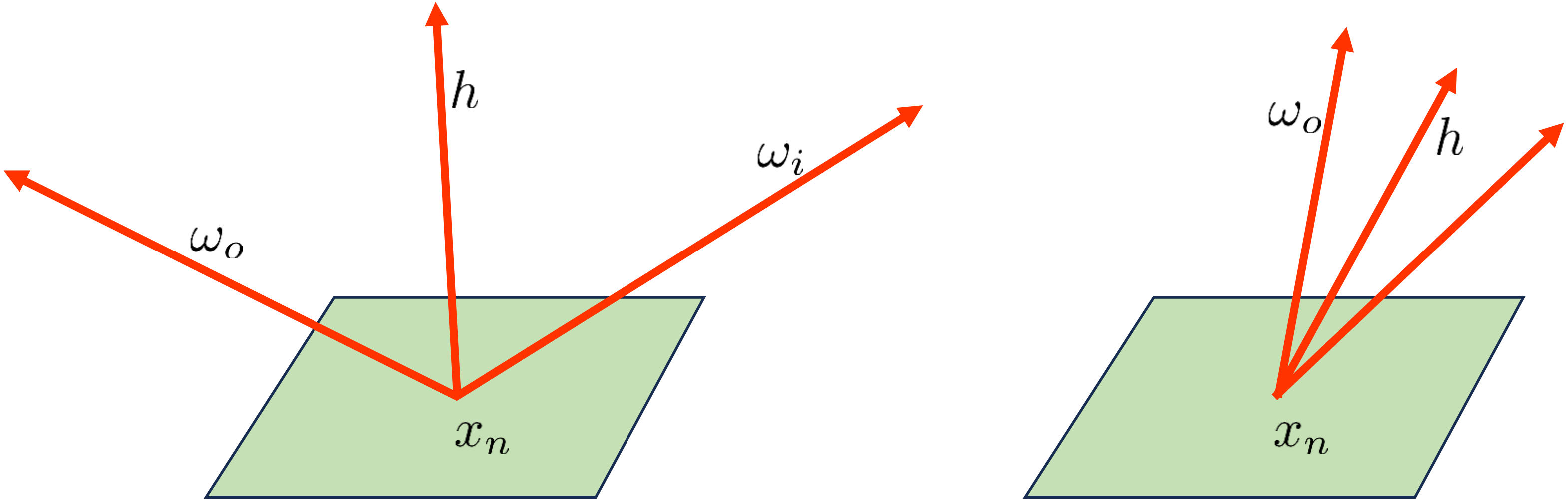

- Functionality
  - Evaluation: $fr(x, \omega_i, \omega_o) = \frac{e+2}{2\pi} \max(0, \omega_r \cdot \omega_i)^e$
  - Sampling:
    - Sample lobe $\theta_{lobe} = \cos^{-1}(\xi_1^{\frac{1}{e+1}})$ <br> $\phi_{lobe} = 2\pi\xi_2$
    - Transform to z-up space $\omega_i = R_{\omega_r}\, \omega_{lobe}$
  - PDF: $p(\omega) = \frac{e+1}{2\pi} \max(0, \omega_r \cdot \omega)^e$

# Glossy: Phong


- Implement this in PlasticBSDF
  - This is a more general PlasticBSDF class
    - Can use more sophisticated models
    - Use Phong as a placeholder
  - Use exponent computed from `alphaToPhongExponent()`
  - Add evaluation code
  - Add sampling and PDF (will need to create `Frame` )

# Glossy: Blinn


- Cosine around the half vector $h = \dfrac{\omega_i + \omega_o}{\|\omega_i + \omega_o\|}$

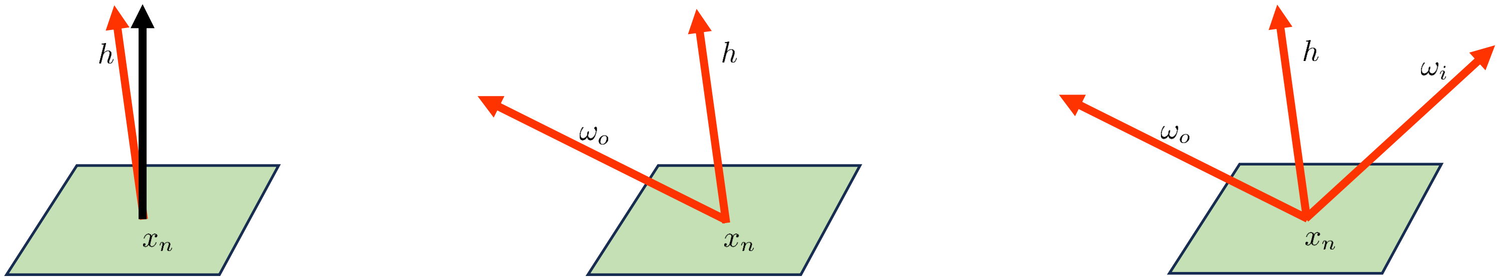


# Glossy: Blinn


- Uses cosine lobe with exponent

$$f_r(x, \omega_i, \omega_o) = \max(0,\ n \cdot h)^e$$

- Sampling needs to sample half vector then reflect $\omega_o$ over $h$



# Glossy: Blinn


- Sampling
  - Sample $\theta_h$ and $\phi_h$ of half vector

$$\theta_h = \arccos\left((1-\xi_1)^{\frac{1}{e+1}}\right)$$
$$\phi_h = 2\pi\,\xi_2$$

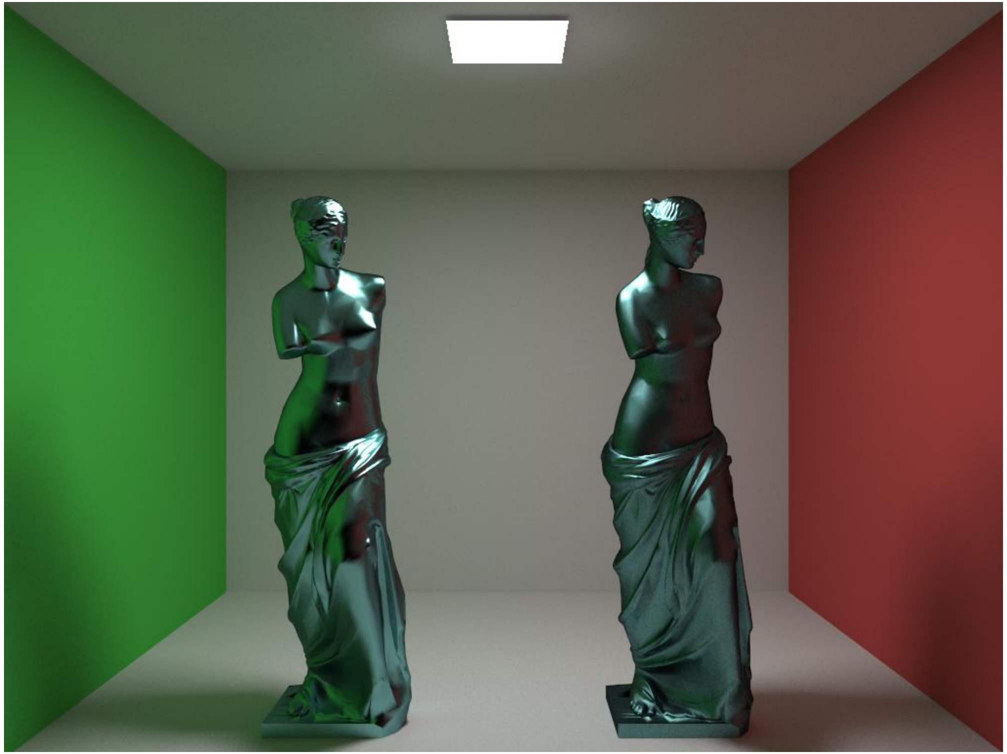
    - Convert to cartesian
  - Reflect over half vector

$$\omega_i = 2\,(\omega_o \cdot h)\,h - \omega_o$$

# Glossy: Blinn


- Functionality
  - Evaluation: $f_r(x, \omega_i, \omega_o) = \max(0,\ n \cdot h)^e$
  - Sampling:
    - Sample half vector $\theta_h = \arccos\left((1-\xi_1)^{\frac{1}{e+1}}\right)$, $\phi_h = 2\pi\,\xi_2$
    - Reflect $\omega_i = 2\,(\omega_o \cdot h)\,h - \omega_o$
  - PDF: $p(\omega_i) = \dfrac{p_h(h)}{4\,(\omega_o \cdot h)}$ where $p_h(h) = \dfrac{e+1}{2\pi}\left(\max(0,\ n \cdot h)\right)^e$

# Glossy: Blinn

# Glossy: LaFortune


- One lobe not expressive enough for many real materials
  - Add more lobes ($lobes$)
  - Add more flexibility to the lobe orientations
  - Add colour to each lobe $C_j$

$$f_r(x,\omega_i,\omega_o) = \sum_{j=1}^{lobes} C_j(\omega_o^T \mathbf{M}_j \omega_i)^{e_j}$$

# Glossy: LaFortune


- Lobe orientations modelled as a matrix $\mathbf{M}_j$
  - Symmetric
  - Only diagonal elements leads to isotropic BSDF
    - Each scales reflected radiance in x, y, and z directions
  - Anisotropy handled by off diagonal elements

# Glossy: LaFortune


- Lobe orientations modelled as a matrix $\mathbf{M}_j$
  - Special case: isotropic, all equal

$$\mathbf{M}_j = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{bmatrix}$$

    - If one lobe then Phong

$$f_r(x, \omega_i, \omega_o) = C\,(m(\omega_o \cdot \omega_i))^e$$

# Glossy: LaFortune


- Lobe orientations modelled as a matrix $\mathbf{M}_j$
  - Special case: isotropic, different diagonal
    - Leads to anisotropic highlights e.g. brushed metal

$$\mathbf{M}_j = \begin{bmatrix} m_x & 0 & 0 \\ 0 & m_y & 0 \\ 0 & 0 & m_z \end{bmatrix}$$

$$f_r(x, \omega_i, \omega_o) = \sum_{j=1}^{lobes} C_j \left( m_{j,x}\omega_{ox}\omega_{ix} + m_{j,y}\omega_{oy}\omega_{iy} + m_{j,z}\omega_{oz}\omega_{iz} \right)^{e_j}$$

# Glossy: LaFortune

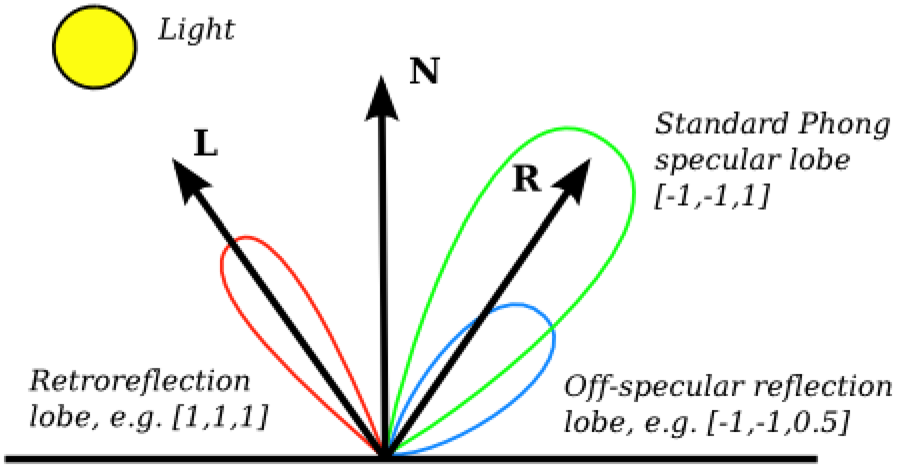

- Sampling
  - Sample lobe from $lobes$
    - Proportional to $w_j = \frac{C_j}{\sum_{k=1}^{lobes} C_k}$
  - Sample direction on lobe similar to Phong

$$\theta = \arccos\left((1-\xi_1)^{\frac{1}{e_j+1}}\right)$$
$$\phi = 2\pi\,\xi_2$$

  - Orient along lobe (lobe direction $d_j = \mathbf{M}_j \omega_o$)

# Glossy: LaFortune

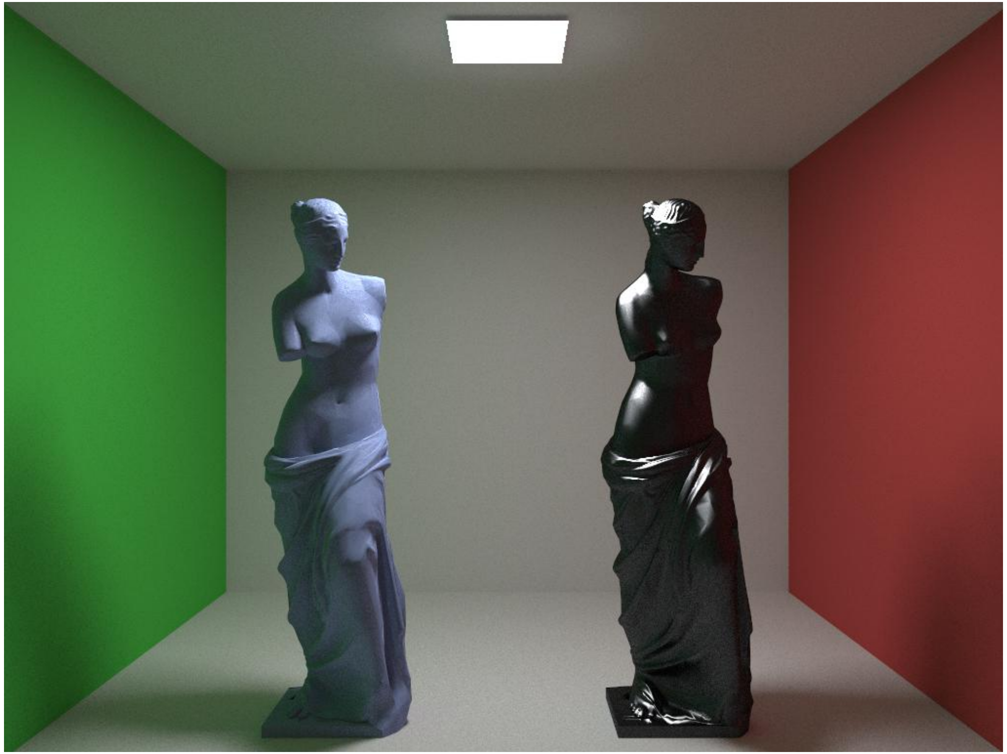

- PDF $$p(\omega_i) = \sum_{j=1}^{lobes} w_j \frac{e_j + 1}{2\pi} (d_j \cdot \omega_j)^{e_j}$$



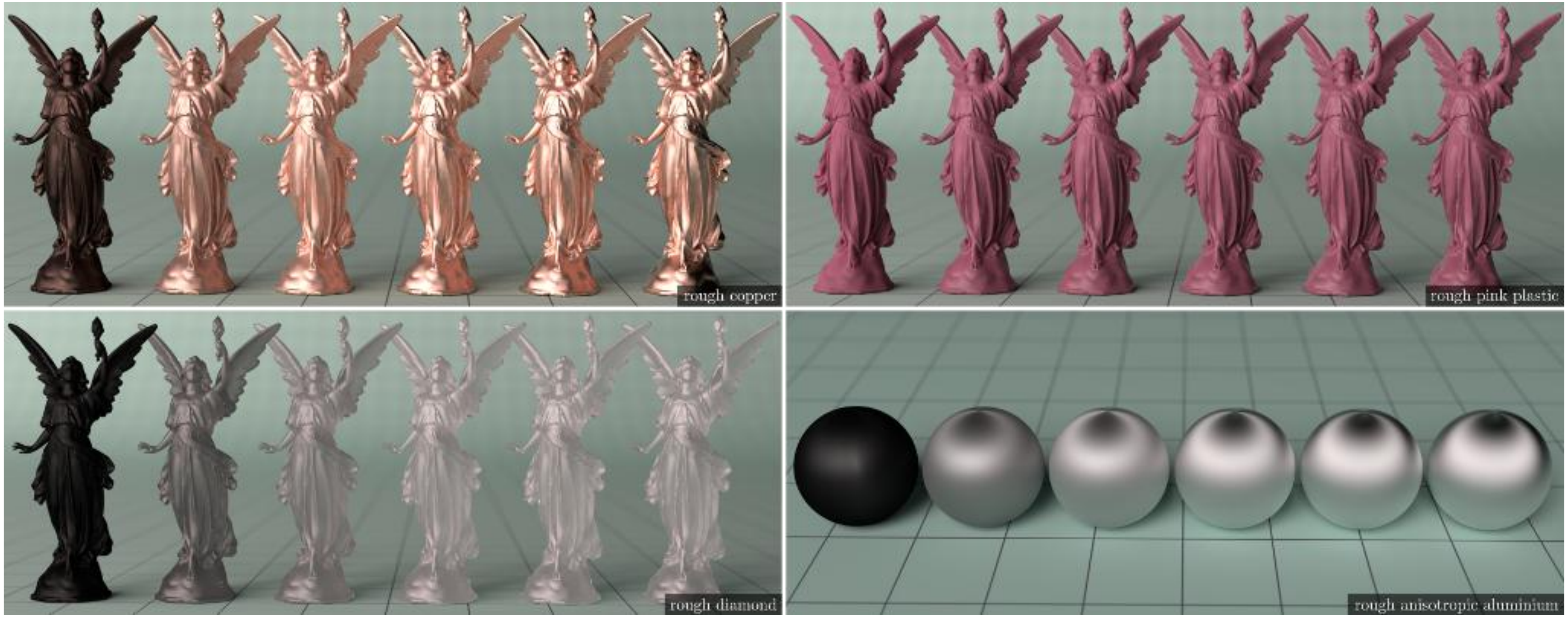
# Glossy: LaFortune


- Functionality
  - Evaluation: $f_r(x, \omega_i, \omega_o) = \sum_{j=1}^{lobes} C_j (\omega_o^T \mathbf{M}_j \omega_i)^{e_j}$
  - Sampling:
    - Sample lobe from $lobes$ $\quad \theta = \arccos\left((1-\xi_1)^{\frac{1}{e_j+1}}\right)$
    - Sample around lobe $\quad \phi = 2\pi\,\xi_2$
      - Transform to z-up space $\quad d_j = \mathbf{M}_j \omega_o$
  - PDF: $p(\omega_i) = \sum_{j=1}^{lobes} w_j \frac{e_j+1}{2\pi} (d_j \cdot \omega_j)^{e_j}$

# Glossy: LaFortune

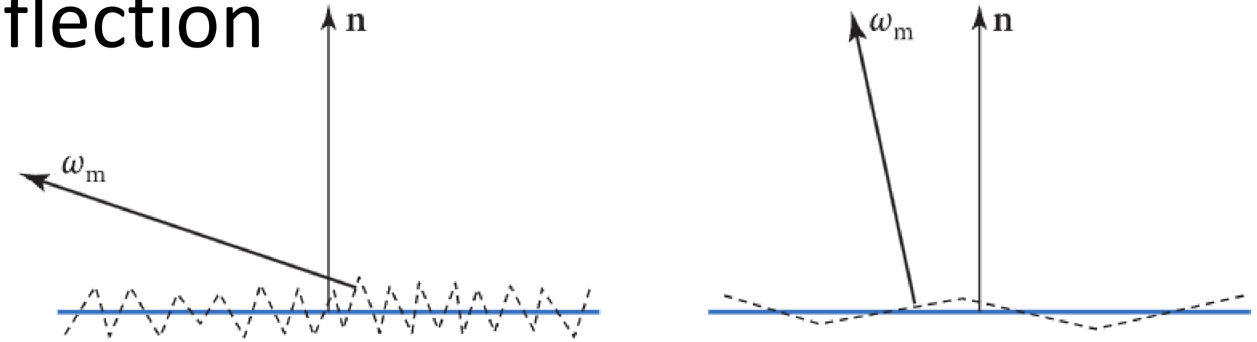
# Glossy: Microfacet

# Glossy: Microfacet

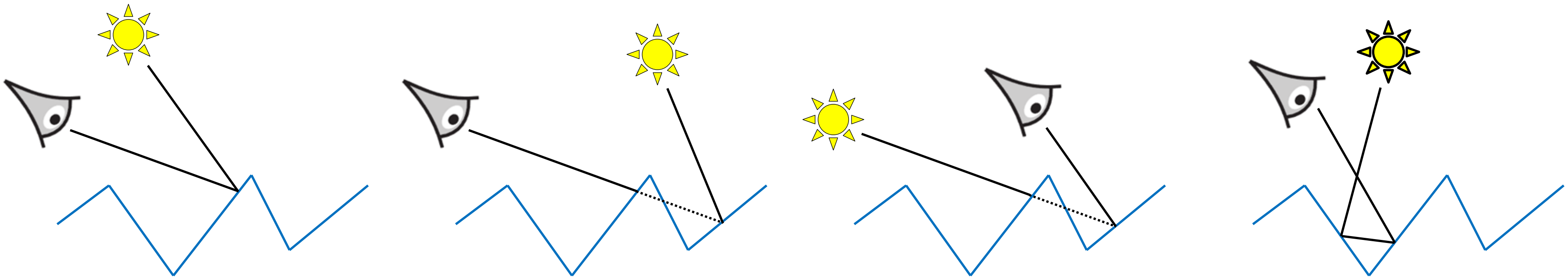

# Glossy: Microfacet


- Surfaces modelled as a distribution of planar facets
- Each facet typically modelled with specular reflection



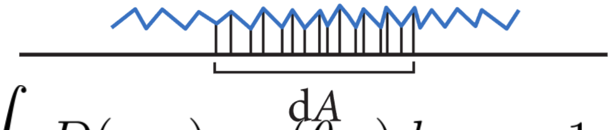
# Glossy: Microfacet


- Lighting effects on microfacet surfaces



# Glossy: Microfacet

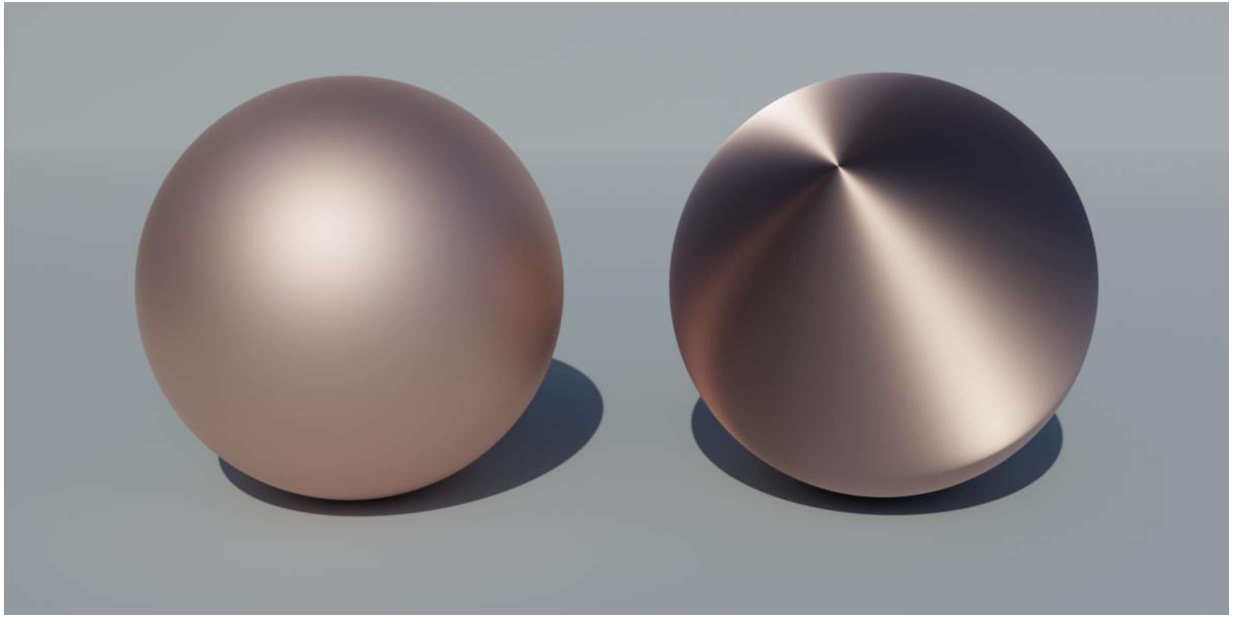

- Specified as a distribution of microfacet normals (Normal Distribution Function): $D(\omega_m)$
  - Microfacet normal: $\omega_m$
  - Must obey $\int_{dA_\mu} (\omega_m(p) \cdot n) dp = \int_{dA} dp$




  - So $\int_\Omega D(\omega_m)(\omega_m \cdot n) d\omega_m = \int_\Omega D(\omega_m) \cos(\theta_m) d\omega_m = 1$

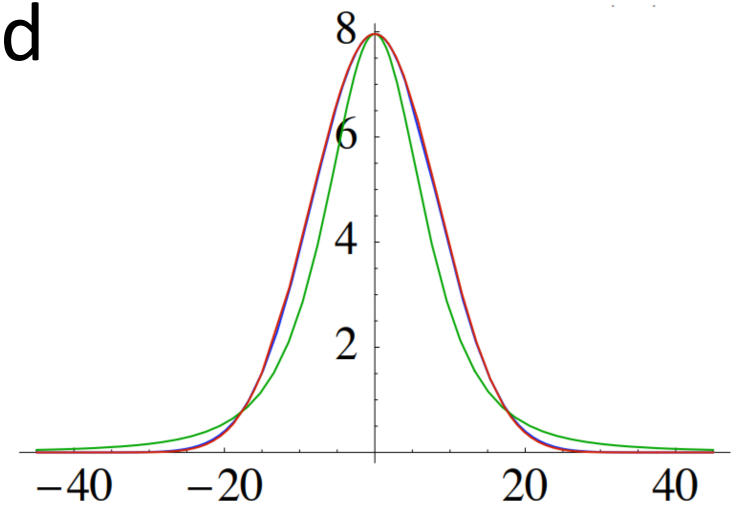
# Glossy: Microfacet


- Isotropic vs Anisotropic



# Glossy: Microfacet

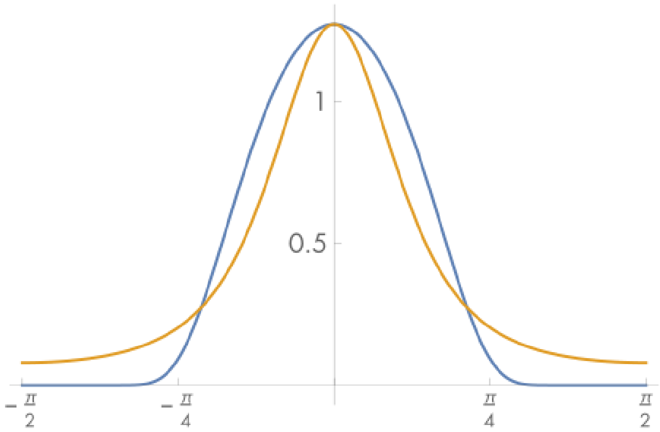

- Several distributions used
  - Trowbridge-Reitz (GGX)
  - Beckmann–Spizzichino
  - Phong
  - Others

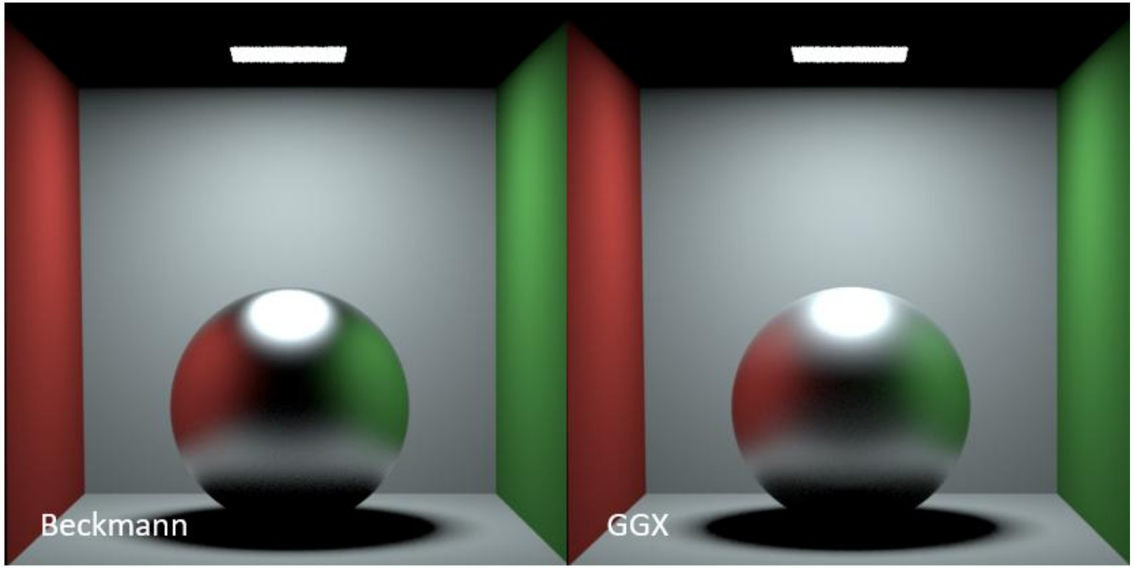



- Different peaks and tails in the distribution

# Glossy: Microfacet

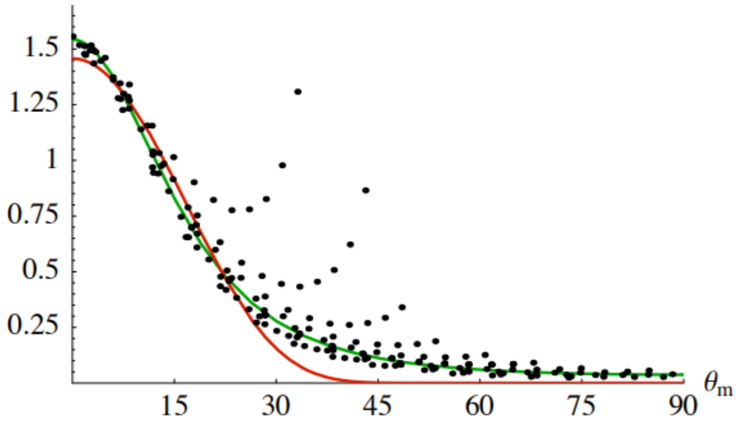

- Differences



# Glossy: Microfacet


- Trowbridge-Reitz (GGX)
  - Distribution follows ellipsoidal shape
  - Good fit to data
  - Anisotropic controlled by two parameters $\alpha_x$ and $\alpha_y$
  - Isotropic $\alpha$



# Glossy: Microfacet


- Trowbridge-Reitz (GGX)
  - Need $\cos(\theta_m) = (\omega_m \cdot n) = H_z$
  - Anisotropic (need $\phi_m$ )

$$D(\omega_m) = \frac{1}{\pi \alpha_x \alpha_y \cos^4(\theta_m) \left(1 + \tan^2(\theta_m) \left(\frac{\cos^2(\phi_m)}{\alpha_x^2} + \frac{\sin^2(\phi_m)}{\alpha_y^2}\right)\right)^2}$$

  - Isotropic $D(\omega_m) = \dfrac{\alpha^2}{\pi \left(\cos^2(\theta_m)(\alpha^2 - 1) + 1\right)^2}$

# Glossy: Microfacet


- Beckmann–Spizzichino
  - Anisotropic

$$D(\omega_m) = \frac{e^{-\tan^2(\theta_m)\left(\frac{\cos^2(\phi_m)}{\alpha_x^2}+\frac{\sin^2(\phi_m)}{\alpha_y^2}\right)}}{\pi\alpha_x\alpha_y\cos^4(\theta_m)}$$

  - Isotropic

$$D(\omega_m) = \frac{e^{-\frac{\tan^2(\theta_m)}{\alpha^2}}}{\pi\alpha^2\cos^4(\theta_m)}$$

# Glossy: Microfacet


- Given a facet, we can compute reflection
  - But overestimate because of masking and shadowing
  - Cannot compute exactly because of distribution, but can model based on assumptions (e.g. v cavity)

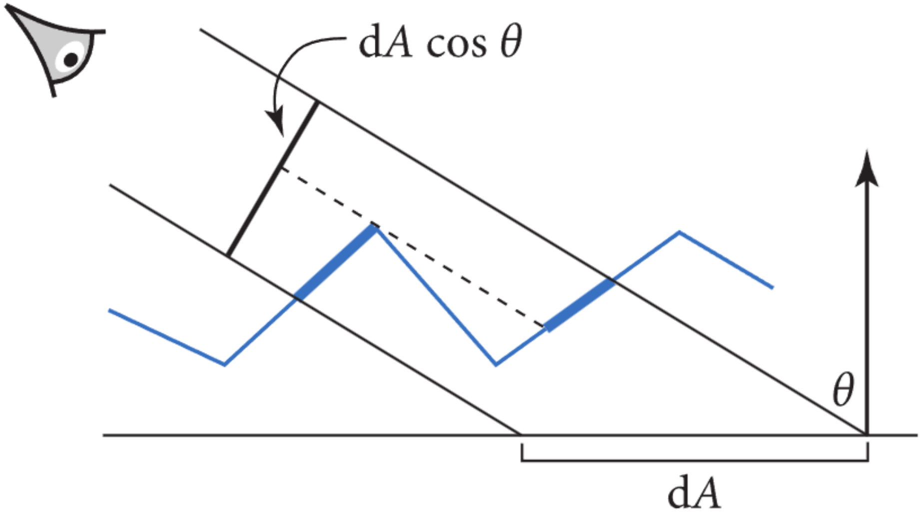
# Glossy: Microfacet


- Masking-Shadowing Function: $G(\omega_o, \omega_i)$
  – Similar problems and can assume statistical independence of masking and shadowing

$$G(\omega_o, \omega_i) = G_1(\omega_o, \omega_m)G_1(\omega_i, \omega_m)$$

# Glossy: Microfacet


- Masking Function: $G_1(\omega, \omega_m)$
  - Fraction of visible area of microfacets to total area



# Glossy: Microfacet


- Masking Function: $G_1(\omega, \omega_m)$
  - Recall $\int_\Omega D(\omega_m)(\omega_m \cdot n) d\omega_m = 1$
  - So considering visible regions of microfacets in a direction $\omega$

$$\int_\Omega D(\omega_m) G_1(\omega, \omega_m) max(0, (\omega \cdot \omega_m)) d\omega_m = (\omega \cdot n) = \cos(\theta)$$

# Glossy: Microfacet


- Masking Function: $G_1(\omega, \omega_m)$
  - Infinite number of functions satisfy this
  - Must make assumptions:
    - Heights and microfacet normals statistically independent
    - Masking is then independent of the microfacet normal
    - So can re-write as

$$G_1(\omega) \int_\Omega D(\omega_m) max(0, (\omega \cdot \omega_m)) d\omega_m = \cos(\theta)$$

# Glossy: Microfacet

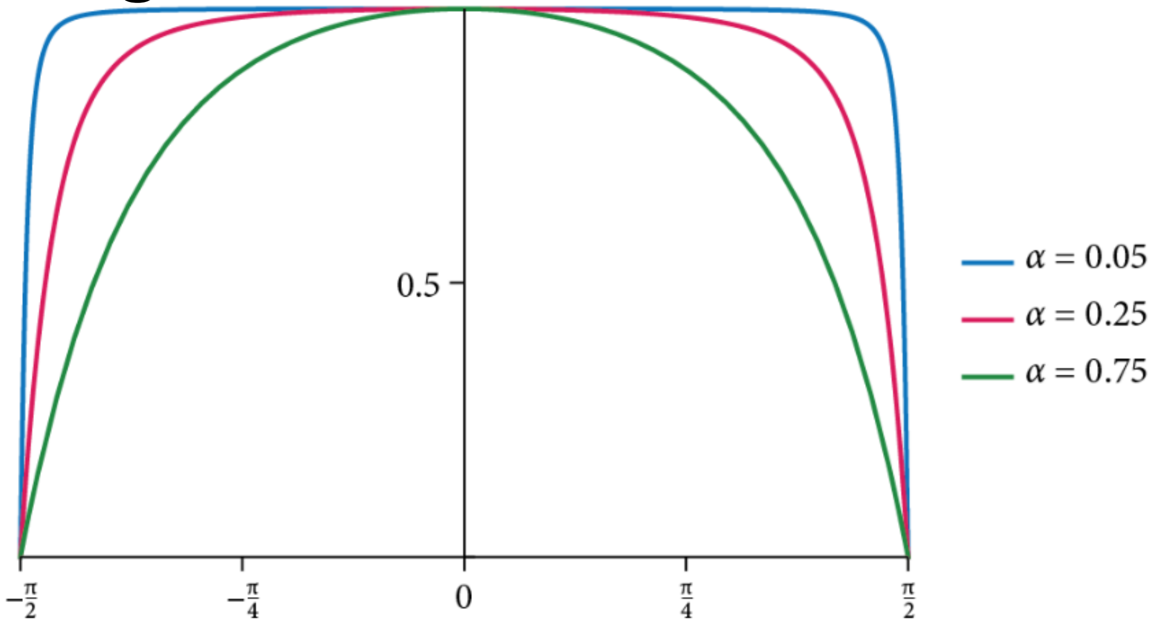

- Masking Function: $G_1(\omega, \omega_m)$
  - Rearrange $G_1(\omega) = \dfrac{\cos(\theta)}{\int_\Omega D(\omega_m) max(0, (\omega \cdot \omega_m)) d\omega_m}$
  - Known as Smiths approximation
    - Works well for real materials
    - Despite simplifications!

# Glossy: Microfacet


- Masking Function: $G_1(\omega)$
  - For Trowbridge-Reitz (GGX): $G_1(\omega) = \dfrac{1}{1+\Lambda(\omega)}$
  - Where: $\Lambda(\omega) = \dfrac{\sqrt{1+\alpha^2 \tan^2(\theta)} - 1}{2}$
  - For anisotropic: $\alpha = \sqrt{\alpha_x^2 \cos^2(\phi_x) + \alpha_y^2 \sin^2(\phi_x)}$

# Glossy: Microfacet


- Masking Function: $G_1(\omega)$



# Glossy: Microfacet


- Masking Function: $G_1(\omega)$
  - For Beckmann $$G_1(\omega) = \frac{1}{1+\Lambda(\omega)}$$
  - Where: $$\Lambda(\omega) = \frac{1}{2}\left(erf\left(\frac{1}{\alpha\tan(\theta)}\right) - 1 + \frac{e^{-\left(\frac{1}{\alpha\tan(\theta)}\right)^2}}{\frac{1}{\alpha\tan(\theta)}\sqrt{\pi}}\right)$$

# Glossy: Microfacet


- So far distribution w.r.t $d\omega_m$ but need w.r.t. $d\omega_i$
- Compute $\frac{d\omega_m}{d\omega_i} = \frac{\sin(\theta_m) d\theta_m d\phi_m}{\sin(\theta_i) d\theta_i d\phi_i}$

$$\omega_m = \frac{\omega_i + \omega_o}{||\omega_i + \omega_o||}$$

$$= \frac{\sin(\theta_m) d\theta_m d\phi_m}{\sin(2\theta_m) 2 d\theta_m d\phi_m}$$

$$\omega_i = -\omega_o + 2(\omega_m \cdot \omega_o)\omega_m$$

$$= \frac{\sin(\theta_m)}{4\cos(\theta_m)\sin(\theta_m)}$$

$$\theta_i = 2\theta_m$$

$$= \frac{1}{4\cos(\theta_m)} = \frac{1}{4(\omega_i \cdot \omega_m)} = \frac{1}{4(\omega_o \cdot \omega_m)}$$

# Glossy: Microfacet


- Microfacet PDF: $p(\omega_i) = p(\omega_m)\dfrac{d\omega_m}{d\omega_i} = \dfrac{D(\omega_m)\cos(\theta_m)}{4(\omega_o \cdot \omega_m)}$
  - $\cos(\theta_m)$ in the numerator from $\int_\Omega D(\omega_m)\cos(\theta_m)d\omega_m = 1$
- Sampling:
  - Sample microfacet normal $\omega_m$ from $D(\omega_m)$
  - Reflect $\omega_o$ over $\omega_m$: $\omega_i = -\omega_o + 2(\omega_m \cdot \omega_o)\omega_m$

# Glossy: Microfacet


- Sampling GGX (will focus on isotropic):
  - Solid angle $$p(\omega_m) = \frac{\alpha^2 \cos(\theta_m)}{\pi \left(\cos^2(\theta_m)(\alpha^2 - 1) + 1\right)^2}$$
  - Spherical coordinates
$$p(\theta_m, \phi_m) = \frac{\alpha^2 \cos(\theta_m) \sin(\theta_m)}{\pi \left(\cos^2(\theta_m)(\alpha^2 - 1) + 1\right)^2}$$
  - Marginal PDF:
$$p(\theta_m) = \int_0^{2\pi} p(\theta_m, \phi_m) d\phi_m = \frac{2\alpha^2 \cos(\theta_m) \sin(\theta_m)}{\left(\cos^2(\theta_m)(\alpha^2 - 1) + 1\right)^2}$$

# Glossy: Microfacet


- Sampling GGX (will focus on isotropic):
  - CDF:

$$CDF(\theta_m) = \int_0^{\theta_m} \frac{2\alpha^2 \cos(t)\sin(t)}{\left(\cos^2(t)(\alpha^2 - 1) + 1\right)^2} dt$$

$$CDF(\theta_m) = \frac{\alpha^2}{\cos^2(\theta_m)(\alpha^2 - 1)^2 + (\alpha^2 - 1)} - \frac{1}{\alpha^2 - 1}$$

  - So

$$\theta_m = \cos^{-1}\left(\sqrt{\frac{1 - \xi_1}{\xi_1(\alpha^2 - 1) + 1}}\right) \qquad \phi_m = 2\pi\xi_2$$

# Glossy: Microfacet


- Sampling Beckmann (will focus on isotropic):
  - Solid angle
    $$p(\omega_m) = \frac{e^{-\frac{\tan^2(\theta_m)}{\alpha^2}} \cos(\theta_m)}{\pi\alpha^2 \cos^4(\theta_m)}$$
  - Spherical coordinates
    $$p(\theta_m, \phi_m) = \frac{e^{-\frac{\tan^2(\theta_m)}{\alpha^2}} \sin(\theta_m)}{\pi\alpha^2 \cos^3(\theta_m)}$$
  - Marginal PDF:

$$p(\theta_m) = \int_0^{2\pi} p(\theta_m, \phi_m) d\phi_m = \frac{2e^{-\frac{\tan^2(\theta_m)}{\alpha^2}} \sin(\theta_m)}{\alpha^2 \cos^3(\theta_m)}$$

# Glossy: Microfacet


- Sampling Beckmann (will focus on isotropic):
  - CDF:

$$CDF(\theta_m) = \int_0^{\theta_m} \frac{2e^{-\frac{\tan^2(t)}{\alpha^2}} \sin(t)}{\alpha^2 \cos^3(t)} dt$$

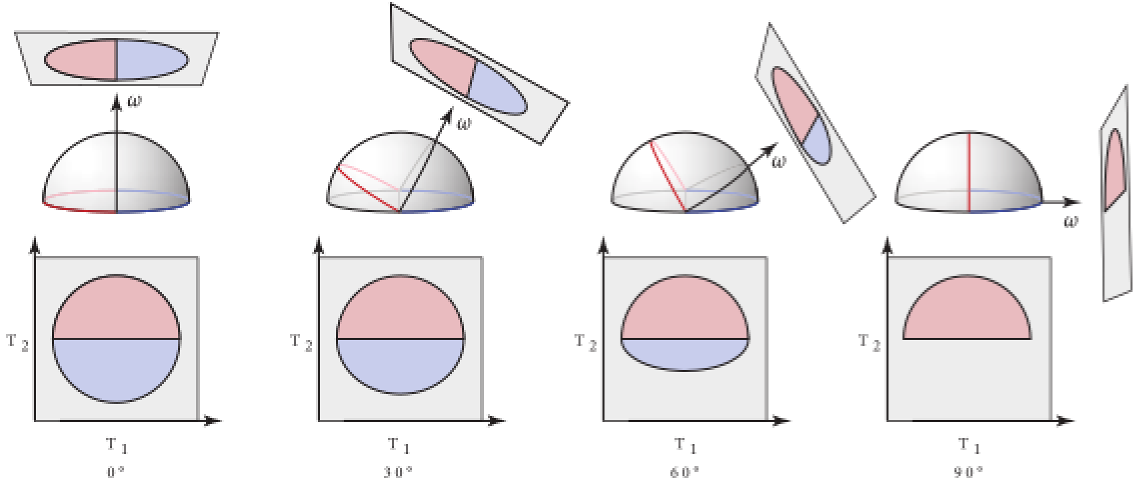
$$CDF(\theta_m) = 1 - e^{\frac{1}{\alpha^2}\left(1 - \frac{1}{\cos^2(\theta_m)}\right)}$$

  - So

$$\theta_m = \cos^{-1}\left(\sqrt{\frac{1}{1 - \alpha^2 ln(1 - \xi_1)}}\right) \qquad \phi_m = 2\pi\xi_2$$

# Glossy: Microfacet


- Sampling Visible Normals:
  - Can sample only visible normal from $\omega_o$



# Glossy: Microfacet


- Sampling Visible Normals:

# Glossy: Microfacet


- Putting it all together (Cook-Torrance BRDF)
  - Masking-Shadowing: $G(\omega_o, \omega_i) = G_1(\omega_o)G_1(\omega_i)$
  - Normal Distribution Function: $D(\omega_m) = D(H.z)$
  - Fresnel: $F(\omega_o)$
  - Light reflected across microfacet $\omega_i = -\omega_o + 2(\omega_m \cdot \omega_o)\omega_m$

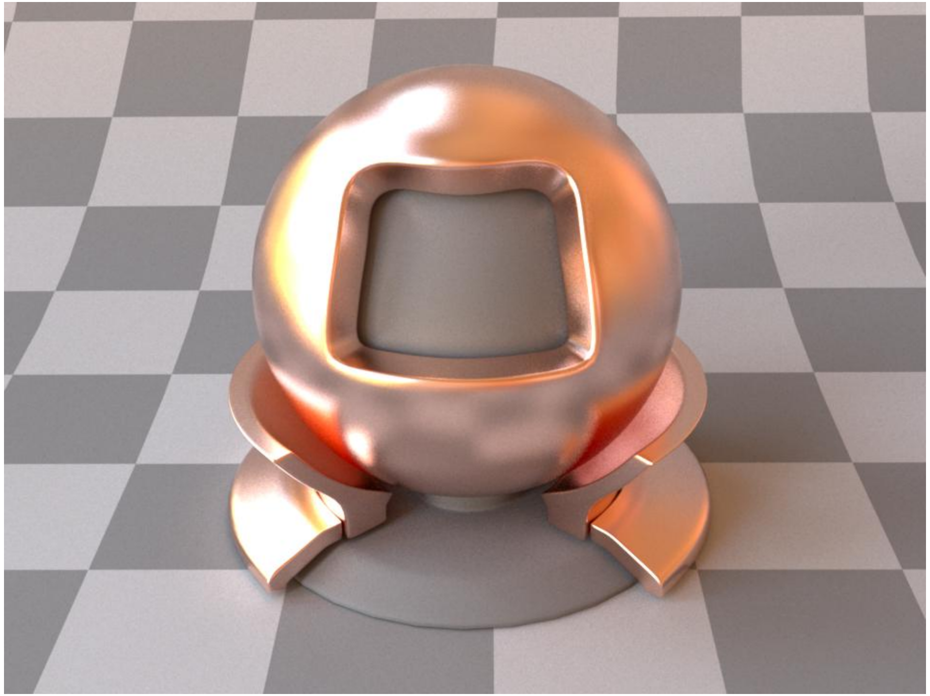
# Glossy: Microfacet


- Putting it all together (Cook-Torrance BRDF)
  - BRDF: $fr(x, \omega_i, \omega_o) = \frac{G(\omega_o, \omega_i) D(\omega_m) F(\omega_o)}{4\cos(\theta_o)\cos(\theta_i)}$
    - Derivation is a bit involved
  - Sampling: $\theta_m = \cos^{-1}\left(\sqrt{\frac{1-\xi_1}{\xi_1(\alpha^2-1)+1)}}\right) \qquad \phi_m = 2\pi\xi_2$

$$\omega_i = -\omega_o + 2(\omega_m \cdot \omega_o)\omega_m$$

  - PDF: $p(\omega_i) = \frac{D(\omega_m)\cos(\theta_m)}{4(\omega_o \cdot \omega_m)}$

# Glossy: Microfacet

# Glossy: Microfacet


- Implementation
  - Add $G(\omega_o, \omega_i)$ and $D(\omega_m)$ in `ShadingHelper`
    - Implement `static float lambdaGGX(Vec3 wi, float alpha)`
    - For use with $G(\omega_o, \omega_i)$
  - Add evaluation to `ConductorBSDF`
    - Then implement sampling and PDF
  - Assignment (optional to add `DielectricBSDF`)

# Glossy: Microfacet


- Implementation
  - If roughness / $\alpha$ < epsilon
    - Then treat as a mirror with Conductor Fresnel

# Microfacet Refraction

- Need to generalize the microfacet normal / half vector to refraction

$$\omega_m = \frac{\eta\omega_i + \omega_o}{||\eta\omega_i + \omega_o||}$$

- And Jacobian for mapping between $\omega_i$ and $\omega_m$

$$d\omega_m = \frac{|\omega_o \cdot \omega_m|}{((\omega_i \cdot \omega_m) + (\omega_o \cdot \omega_m)/\eta)^2} d\omega_i$$

# Microfacet Refraction

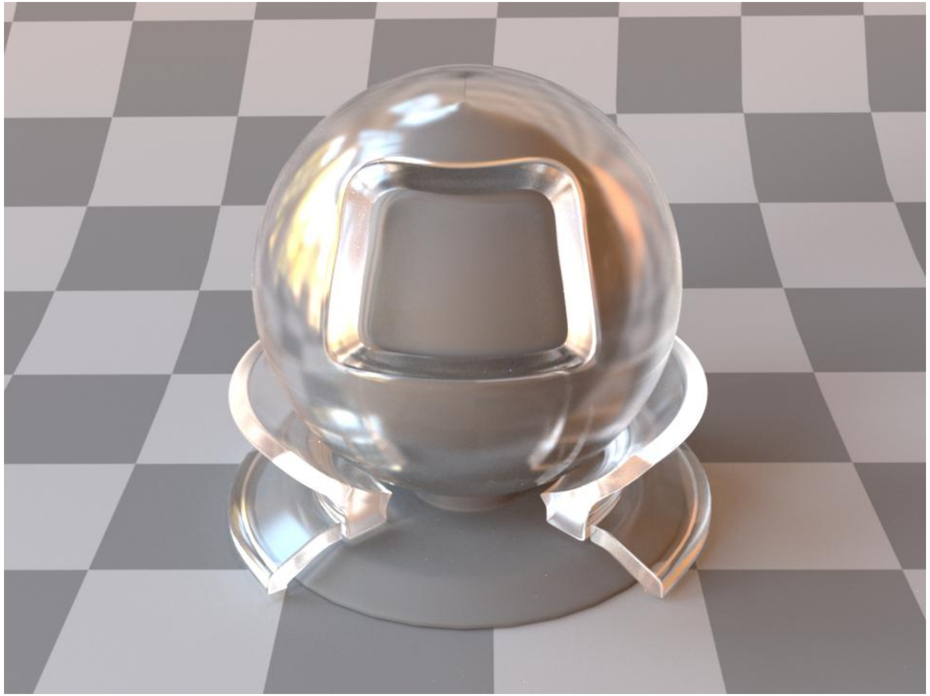
- Define PDF as before

$$p(\omega_i) = p(\omega_m)\frac{d\omega_m}{d\omega_i} = \frac{D(\omega_m)|\omega_o \cdot \omega_m|}{((\omega_i \cdot \omega_m) + (\omega_o \cdot \omega_m)/\eta)^2}$$

- And BSDF

$$fr(x, \omega_i, \omega_o) = \frac{G(\omega_o, \omega_i)D(\omega_m)(1 - F(\omega_o \cdot \omega_m))}{((\omega_i \cdot \omega_m) + (\omega_o \cdot \omega_m)/\eta)^2}\frac{(\omega_i \cdot \omega_m)}{|\cos(\theta_i)|}\frac{(\omega_o \cdot \omega_m)}{|\cos(\theta_o)|}$$

# Microfacet Refraction



# Glossy: Microfacet


- Implementation
  - Assignment
  - Suggest implement ConductorBSDF first
  - DielectricBSDF if time

# Diffuse: Microfacet

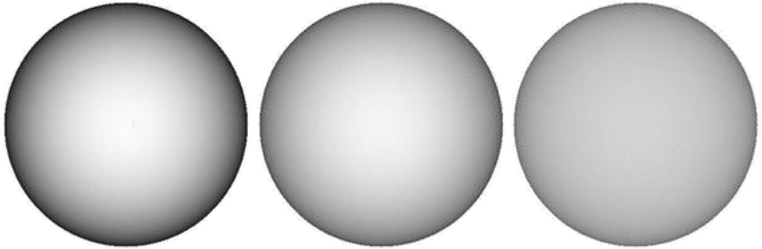

- So far assumed microfacets are specular
- But can be modelled as other functions
- Diffuse: Oren-Nayar
  - Modelled as microfacets with V shape
    - Described by a spherical gaussian distribution

# Diffuse: Microfacet


- Oren-Nayar:
  - Observation: Real objects are not perfect diffuse

# Diffuse: Microfacet


- Oren-Nayar
  - Accounts for interreflections between facets (only one needed due to V shape)
  - Takes a roughness parameter $\sigma$
    - Standard deviation of the surface slopes



$\sigma = 0$ $\sigma = 0.1$ $\sigma = 0.3$

# Diffuse: Microfacet


- Oren-Nayar
  - No closed form expression
  - A good approximation:

$$fr(x,\omega_i,\omega_o) = \frac{\rho(x)}{\pi}\,(A + B\,max(0, \cos(\phi_i - \phi_o)\sin(\max(\theta_i,\theta_o))\tan(\min(\theta_i,\theta_o)))$$

$$A = 1 - \frac{\sigma^2}{2(\sigma^2 + 0.33)}$$

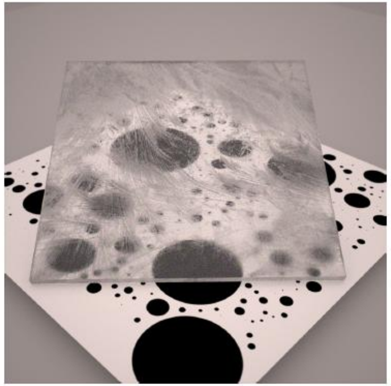
$$B = \frac{0.45\sigma^2}{\sigma^2 + 0.09}$$

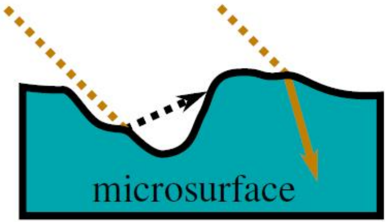
# Diffise: Microfacet

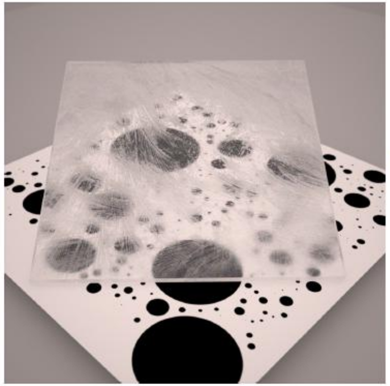

- Implement in OrenNayerBSDF class
  - Implement Oren-Nayer in evaluate
  - Use cosine weighted sampling and PDF
    - Works because Oren Nayer is close to diffuse

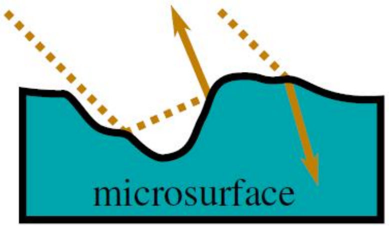
# Microfacet Multiple Scattering

- So far assumed one interaction with microfacets
- But in reality multiple interactions



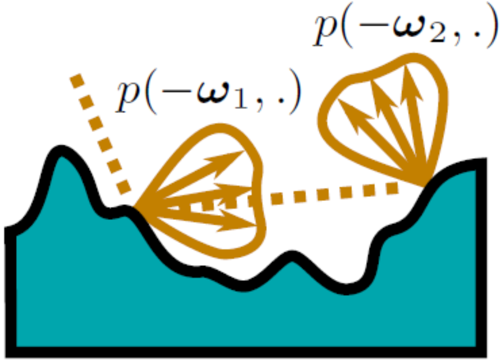




# Microfacet Multiple Scattering

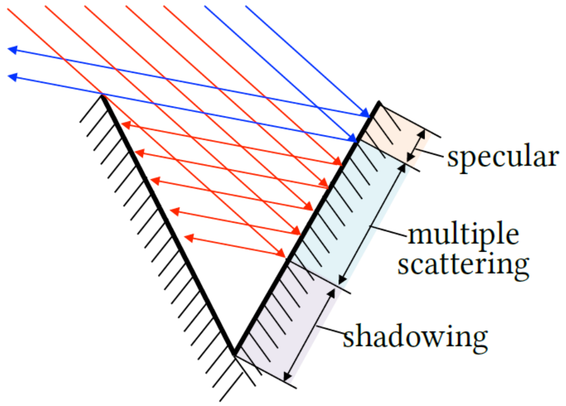
- Option 1: Treat surface as participating media
  - Monte Carlo estimate (noisy *evaluation* of BSDF)

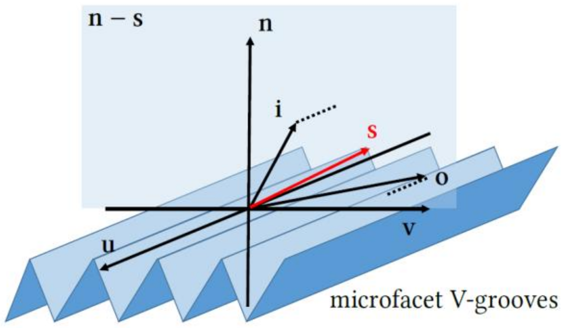


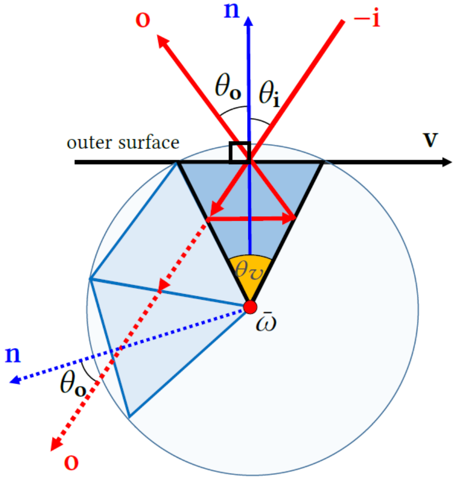
# Microfacet Multiple Scattering

# Microfacet Multiple Scattering

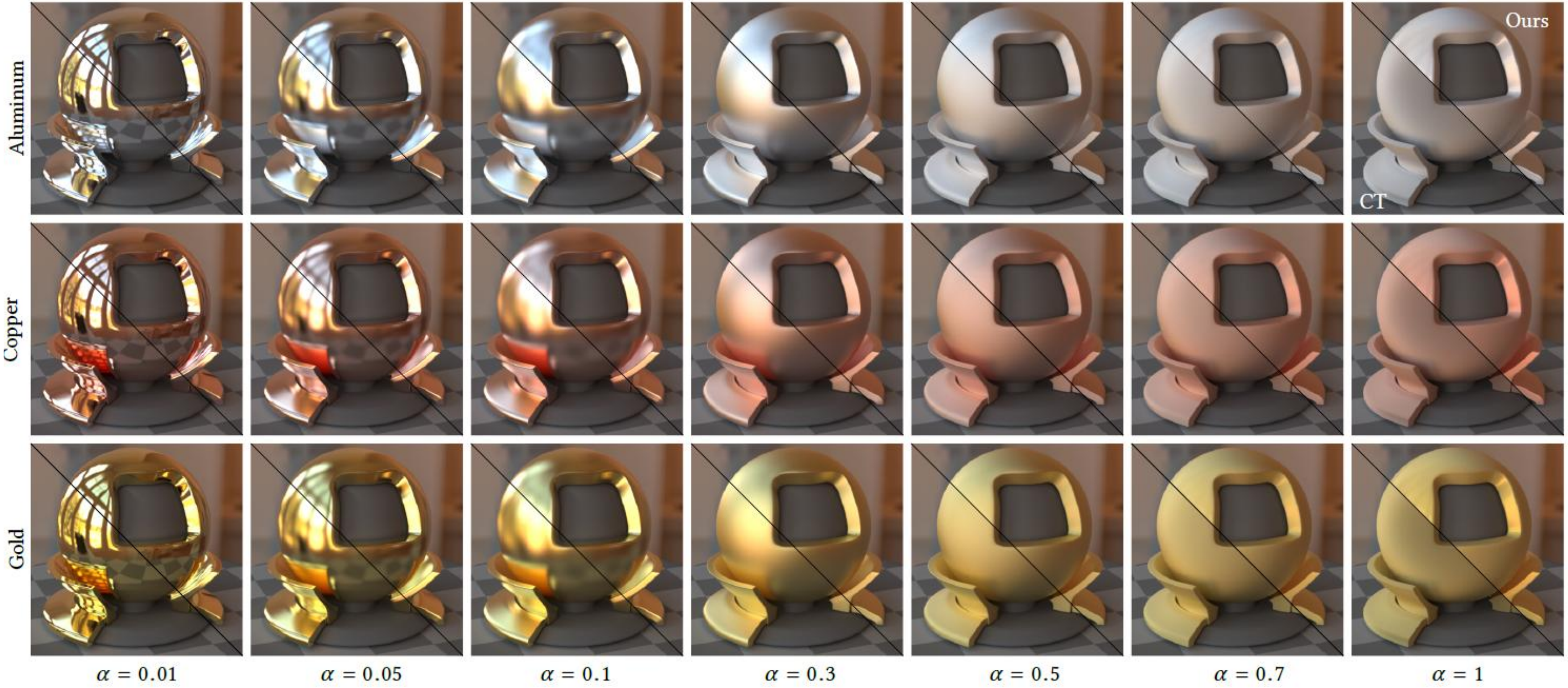
- Option 2: Simulate specular scattering chain in V Grooves



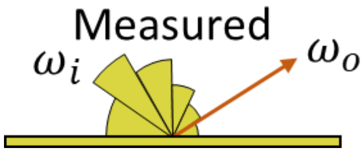



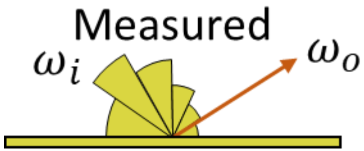
# Microfacet Multiple Scattering



# Measured



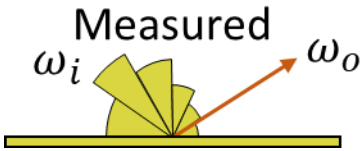

- Use real reflectance data




# Measured

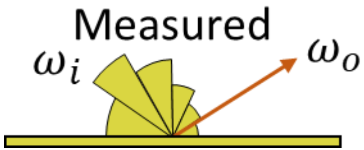
- MERL (Matusik)

# Measured

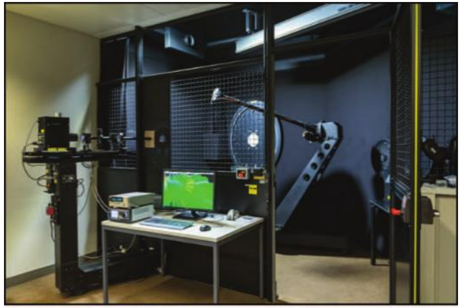


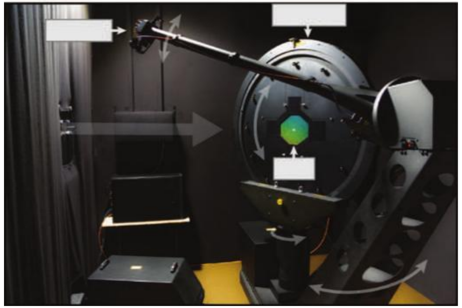

- Dupuy and Jakob

# Measured

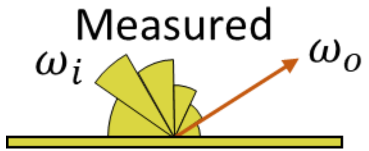



- Captured by specialized hardware

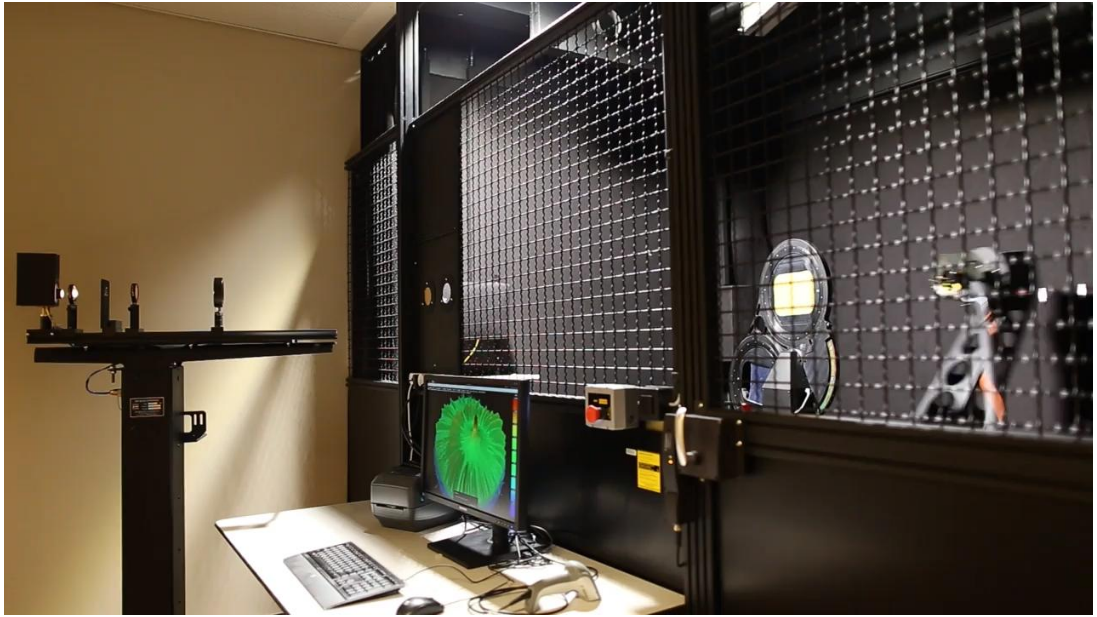




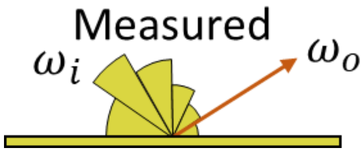
# Measured



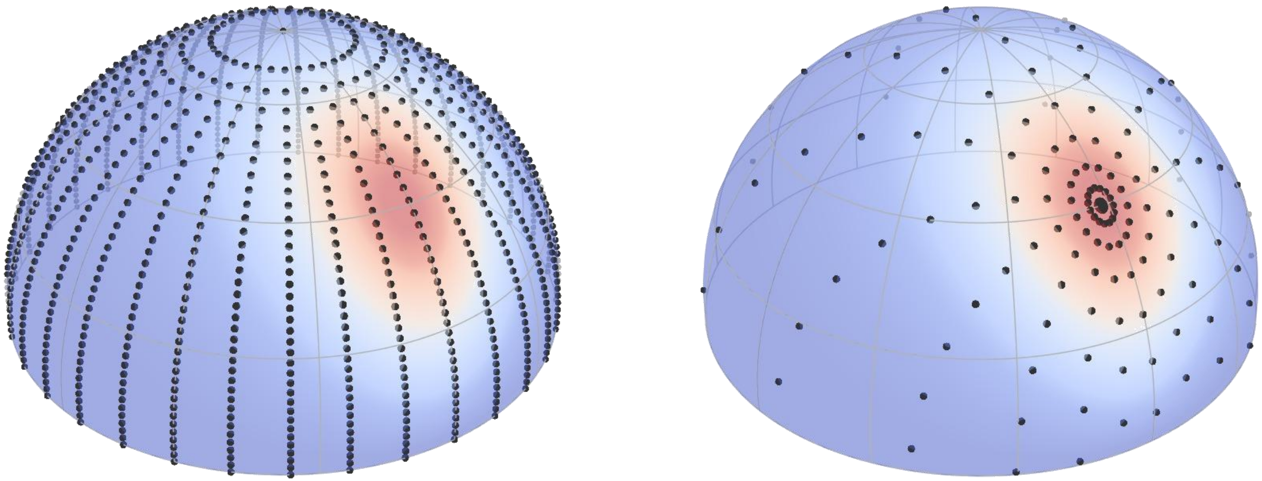

- Captured by specialized hardware



# Measured




- Store in a table / hierarchically
  – Interpolate between samples



# Subsurface Scattering

- Light scattering inside an mesh
- Entrance and exit in different places

# Subsurface Scattering

- Multiple options
  - Fill mesh with participating media
  - Diffusion approximation

# Subsurface Scattering

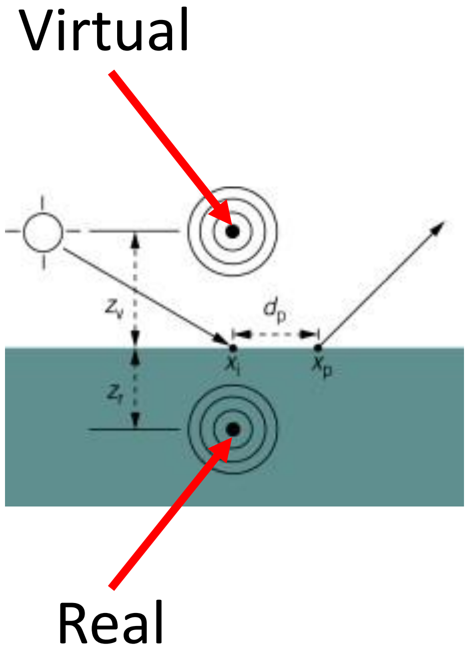
- Volumetric Path Tracing
  - Sample transmissive BSDF at surfa
  - Random walk in medium
  - Until leaves mesh

# Subsurface Scattering

- Diffusion Approximation
  - Makes assumptions about surface
    - Semi infinite
    - Many scattering events $\sigma_s >> \sigma_a$
  - Function of entrance and exit positions and directions $S(x_i, \omega_i; x_o, \omega_o) = \frac{1}{\pi} R(\|x_i - x_o\|) F_r(\omega_o) \cos\theta_o$
    - Where $R(\|x_i - x_o\|)$ is a diffusion profile

# Subsurface Scattering

- Diffusion Approximation
  - Dipole model uses virtual light sources with analytical solution for diffusion profile
  - Real and virtual light source



# Subsurface Scattering

- Diffusion Approximation
  - Diffusion Profile $R(r) = \frac{1}{4\pi D}\left(\frac{e^{-\sigma_{tr} d_r}}{d_r} + A\frac{e^{-\sigma_{tr} d_v}}{d_v}\right)$
  - Effective transport $\sigma_{tr} = \sqrt{3\sigma_a \sigma_t}$
  - Diffusion coefficient $D = \frac{1}{3\sigma_t'}$, $\sigma_t' = \sigma_a + \sigma_s'$, $\sigma_s' = (1-g)\sigma_s$
  - $d_r$ and $d_v$ are distances to dipole light sources

$$d_r = \sqrt{r^2 + z_r^2}, \quad d_v = \sqrt{r^2 + z_v^2} \qquad z_r = \frac{1}{\sigma_t'} \qquad z_v = z_r + 2AD$$

# Subsurface Scattering

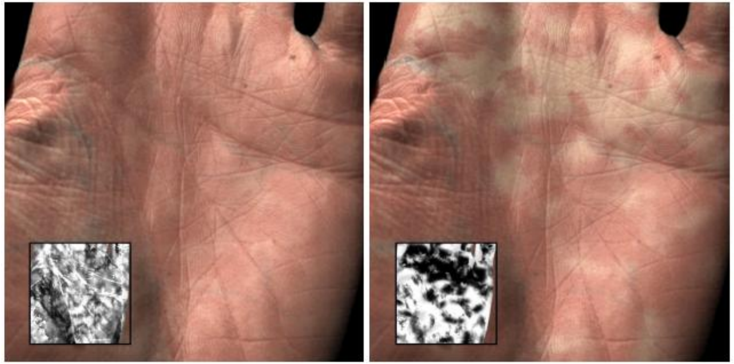
- Diffusion Approximation
  - $A$ is a scaling factor which accounts for Fresnel

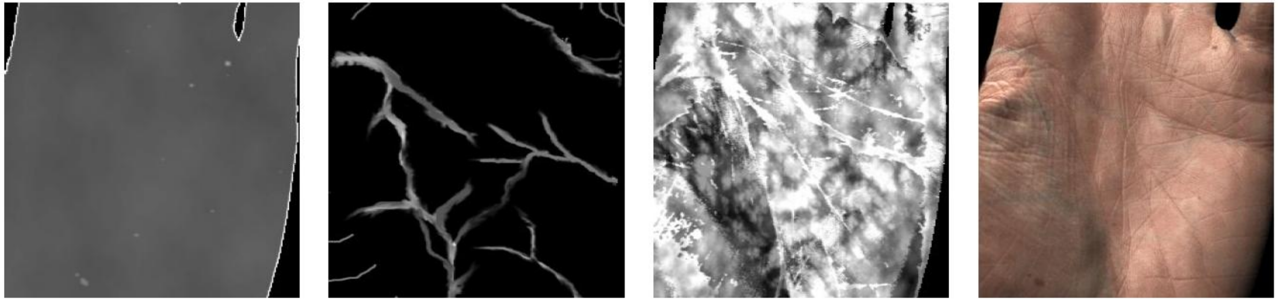
$$A = \frac{1 + F_d}{1 - F_d}$$

- Where $F_d = 1 - \frac{1}{2}\left(\frac{\eta - 1}{\eta + 1}\right)^2 \left(1 + \frac{4}{3}\left(\frac{1}{\eta + 1}\right)\right)$
- is diffuse Fresnel Reflectance
  - How much light from all directions is reflected back into the medium)

# Subsurface Scattering

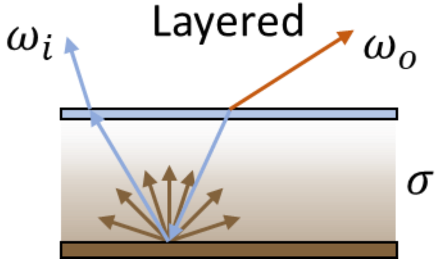
# Subsurface Scattering





Melanin (top layer) | Inter-layer absorption | Hemoglobin (bottom layer) | Final image

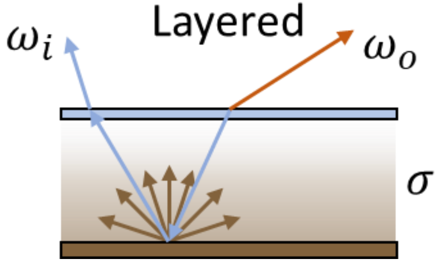
# Layered Materials



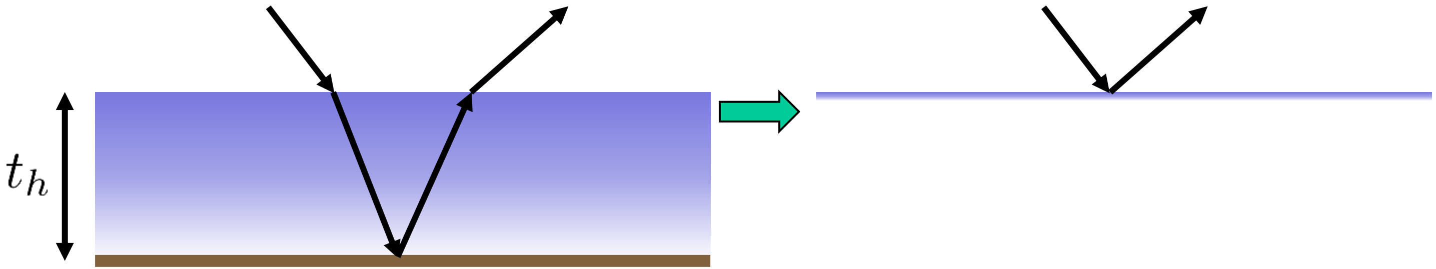

- Want to simulate surfaces with layers
  - Varnish
  - Coatings
  - Car paint
  - Plastics

- Used extensively in films

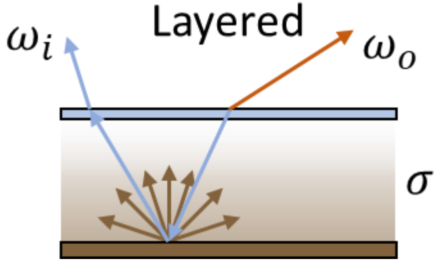
# Layered Materials



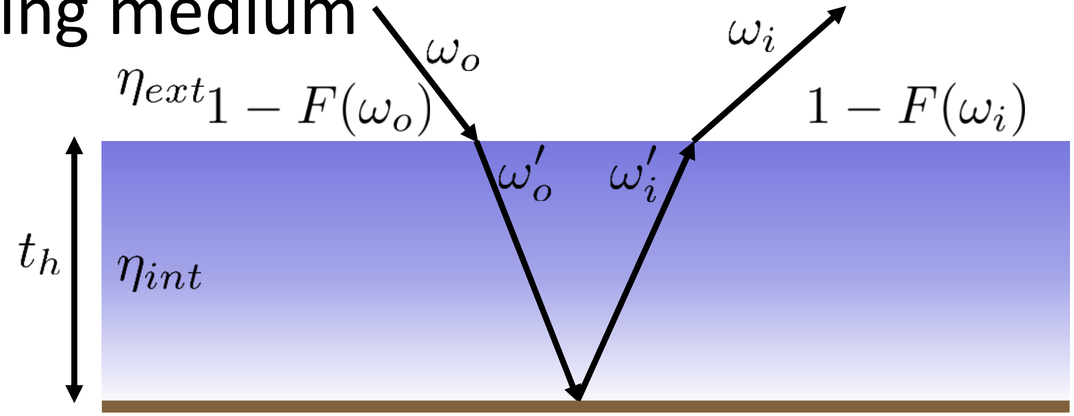

- Simulate light path through interfaces
- All interactions assumed to occur at a single point
- Each layer has a thickness $t_h$



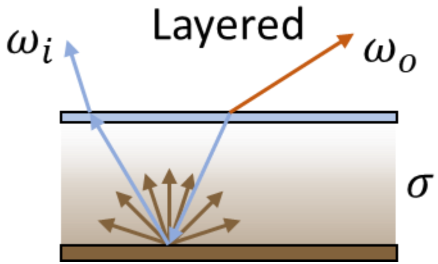
# Layered Materials




- Start with one layer
  - Specular interface
  - Consider absorbing medium
  - Evaluation:
  - Refract $\omega_i$ and $\omega_o$
  - Evaluate Fresnel



# Layered Materials

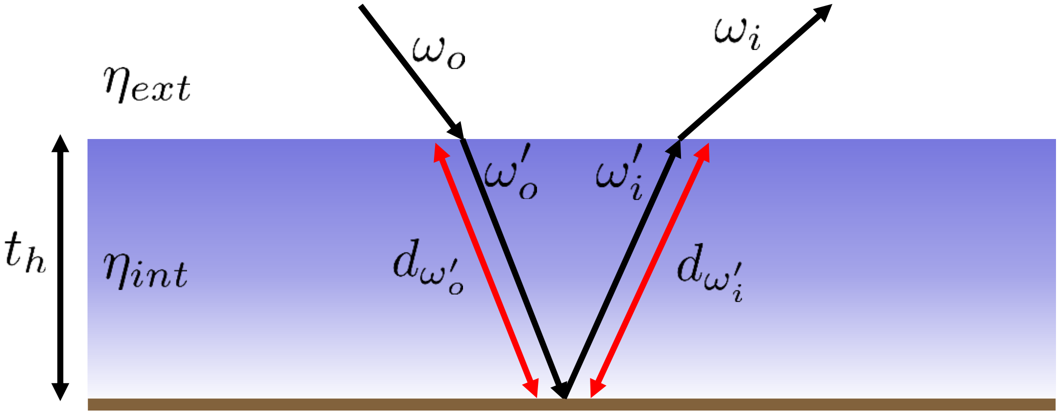



- Start with one layer
  - Evaluation
  - Handle medium

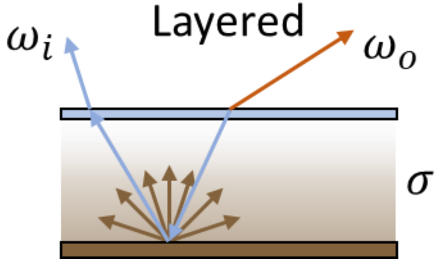
$$d_{\omega'_o} = \frac{t_h}{|((\omega'_o \cdot n))|} \qquad d_{\omega'_i} = \frac{t_h}{|((\omega'_i \cdot n))|}$$

$$T_r(\omega'_o) = e^{-d_{\omega'_o}\sigma_t}$$

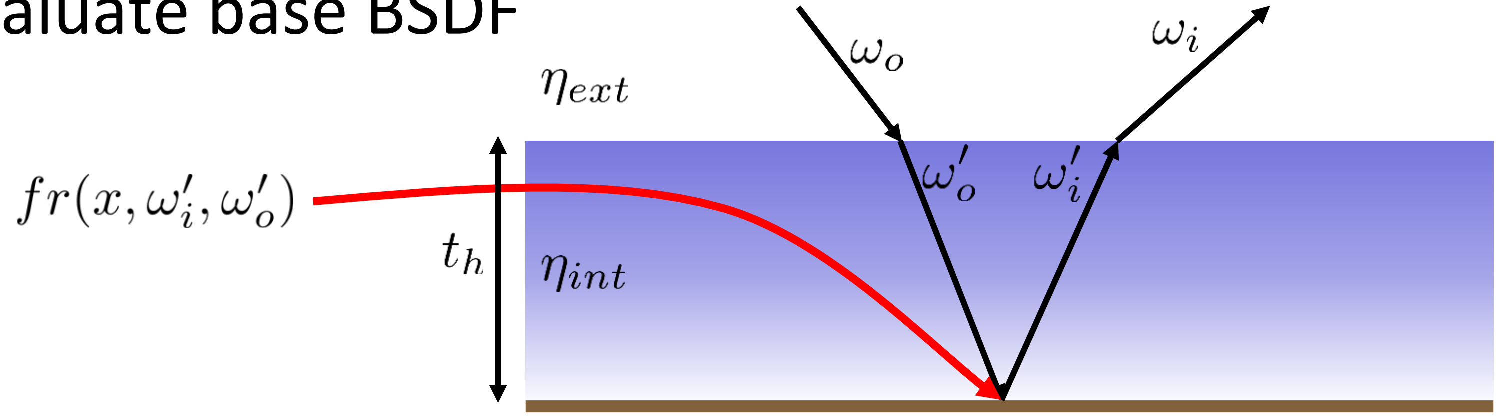
$$T_r(\omega'_i) = e^{-d_{\omega'_i}\sigma_t}$$

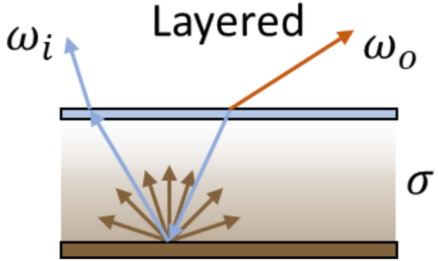


# Layered Materials




- Start with one layer
  - Evaluation
  - Evaluate base BSDF

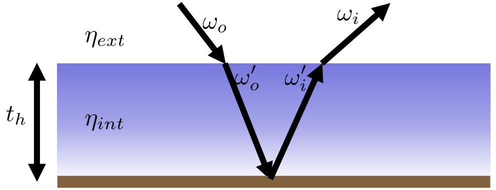





# Layered Materials

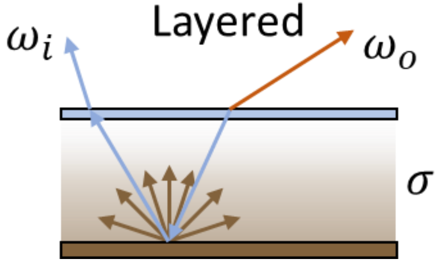
- Start with one layer
  - Evaluation: Put it together

$$fr(x,\omega_i,\omega_o) = \frac{1}{\eta}\frac{(1-F(\omega_i))}{|(\omega_i\cdot n)|}T_r(\omega_i')$$
$$fr(x,\omega_i',\omega_o')$$
$$T_r(\omega_o')\frac{1}{\eta}\frac{(1-F(\omega_o))}{|(\omega_i'\cdot n)|}$$

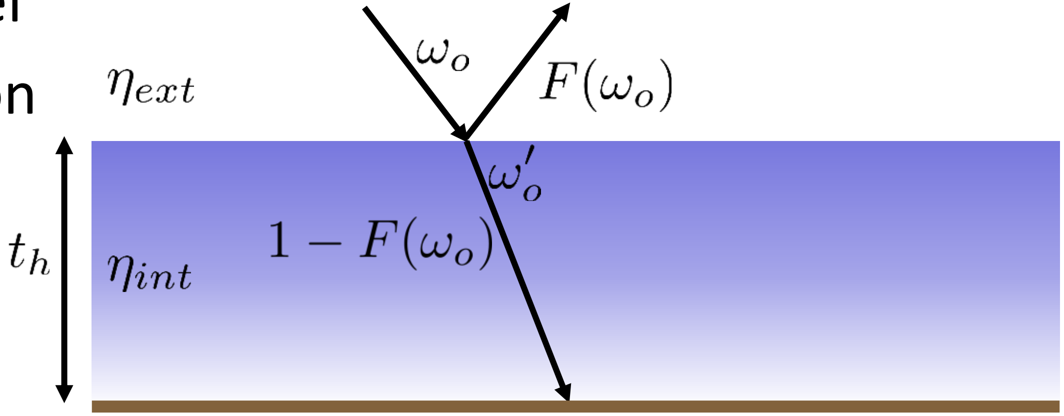



  - Misses specular reflection from interface
    - Needs to be sampled

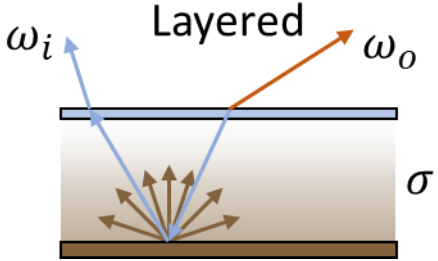
# Layered Materials




- Start with one layer
  - Sampling:
    - Calculate Fresnel
    - Sample reflection
    - Or refraction

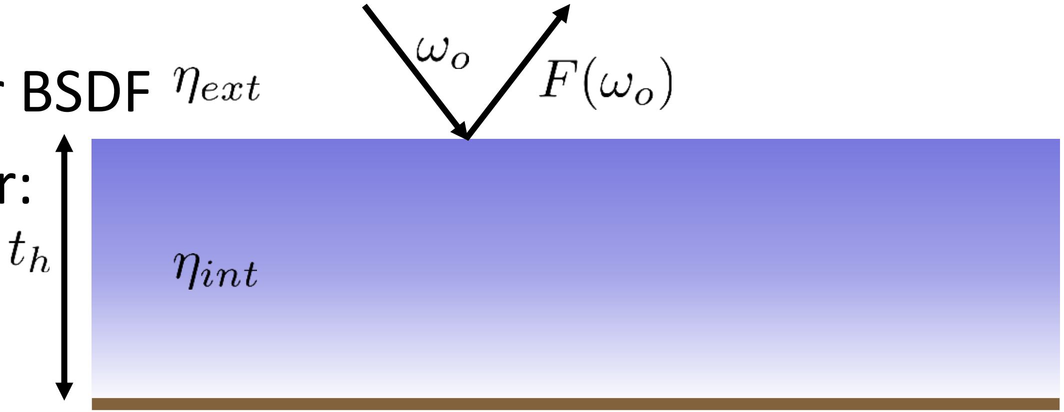


# Layered Materials

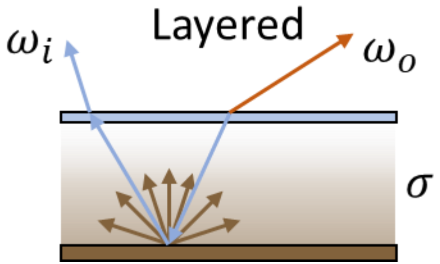



- Start with one layer
  - Sampling:
    - If reflection
    - Sample specular BSDF
    - Evaluated colour:

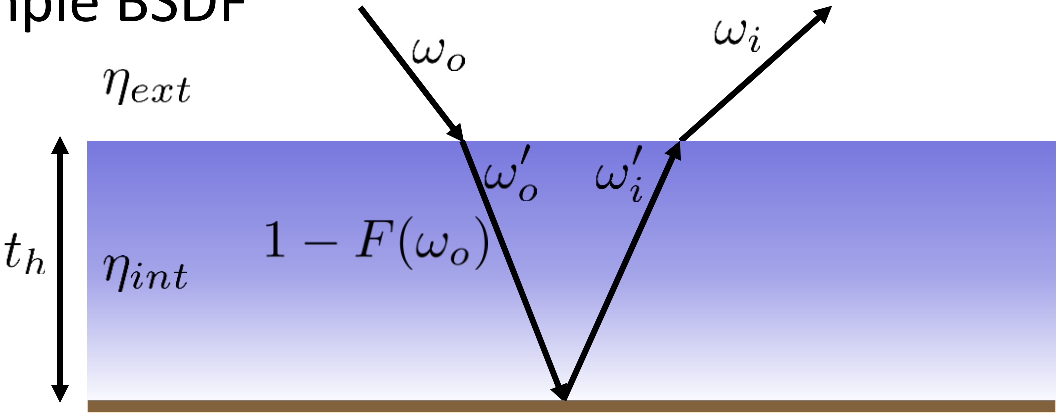
$$\frac{F(\omega_o)}{\cos(\theta)}$$



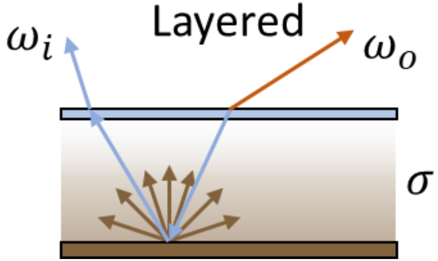
# Layered Materials




- Start with one layer
  - Sampling:
    - If refraction, sample BSDF
    - For sampling
      - Use $\omega_o'$

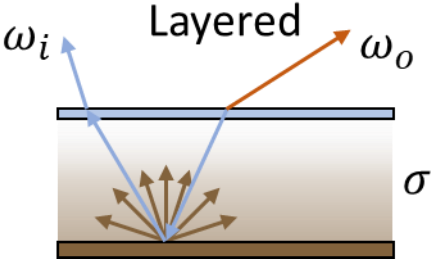


# Layered Materials

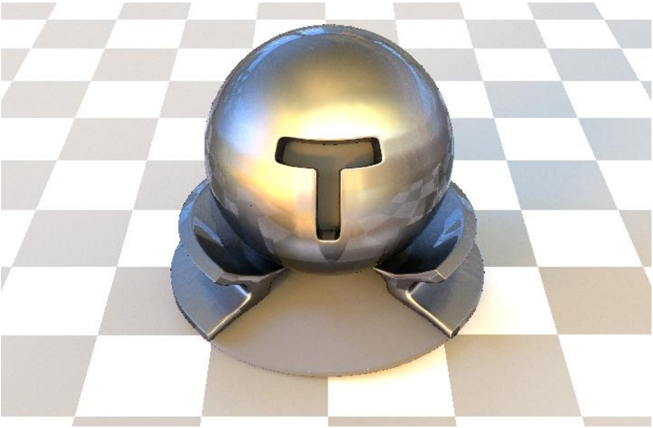



- Start with one layer
- Can simulate many materials
  - Neglects total internal reflection
  - Energy loss
    - More noticable at grazing angles

# Layered Materials

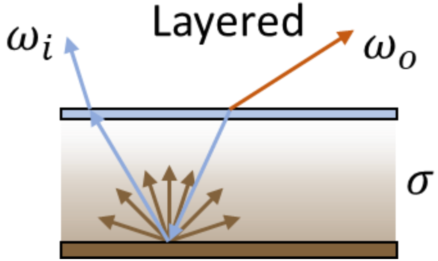


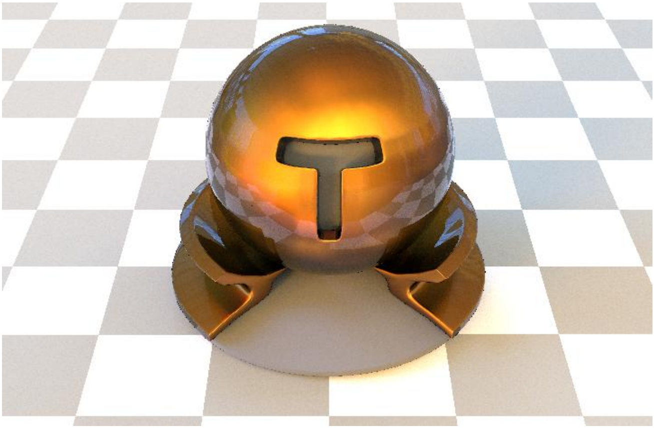

- Different base BSDF



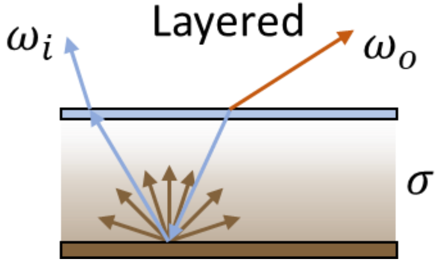
# Layered Materials

- Different base BSDF with tint



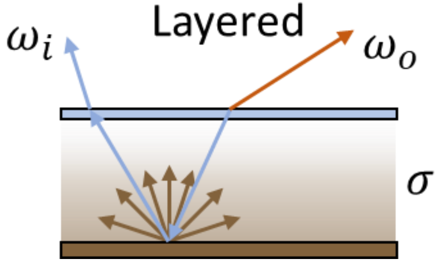



# Layered Materials

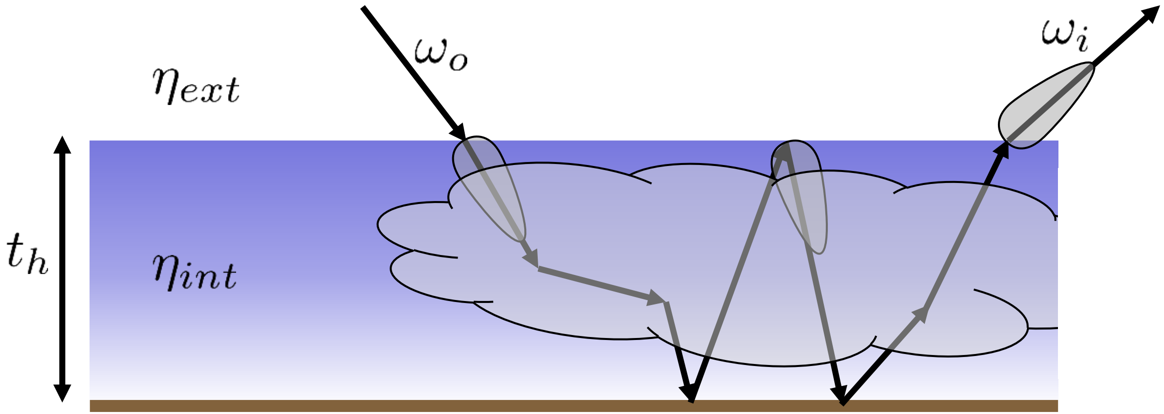



- Can generalize to rough interface
  - Need to sample microfacet on interface
- Can include scattering media and correct reflections at interfaces
  - Monte Carlo estimate of BSDF and PDF
  - More variance, but more accurate images

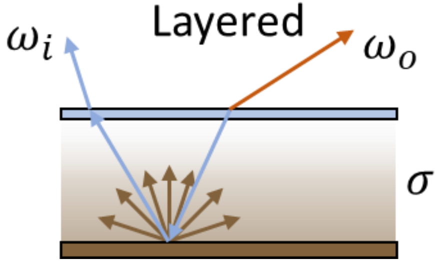
# Layered Materials



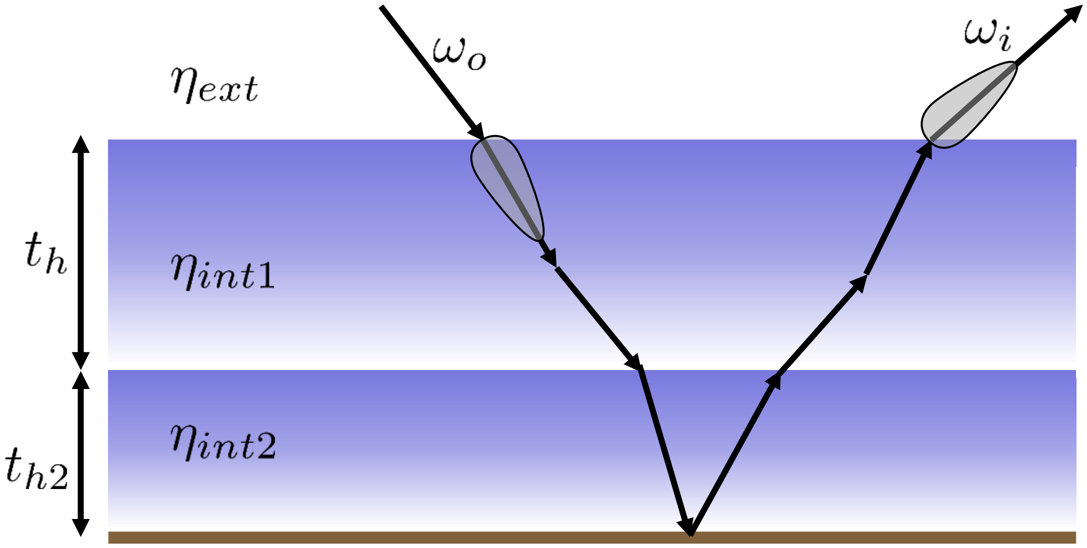

- Can simulate realistic phenomena



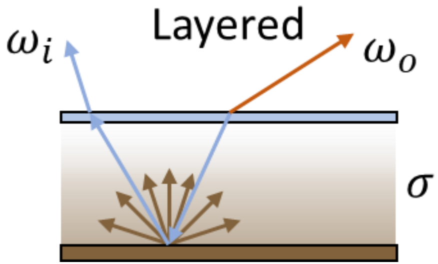
# Layered Materials




- Or multiple layers

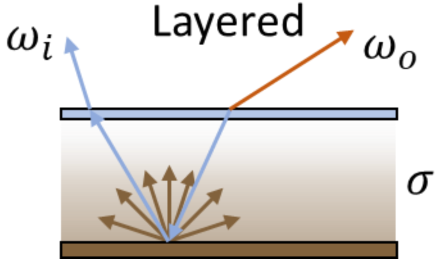


# Layered Materials




- Implementation
  - Option to implement in LayeredBSDF
  - Stores one layer
    - Absorption only
    - thickness stores $t_h$

# Layered Materials




Diffuse

Specular Layer

Glossy Layer

# Blending BSDFs

- Can blend BSDFs based on weights $\sum_{i=1}^{N} w_i = 1$

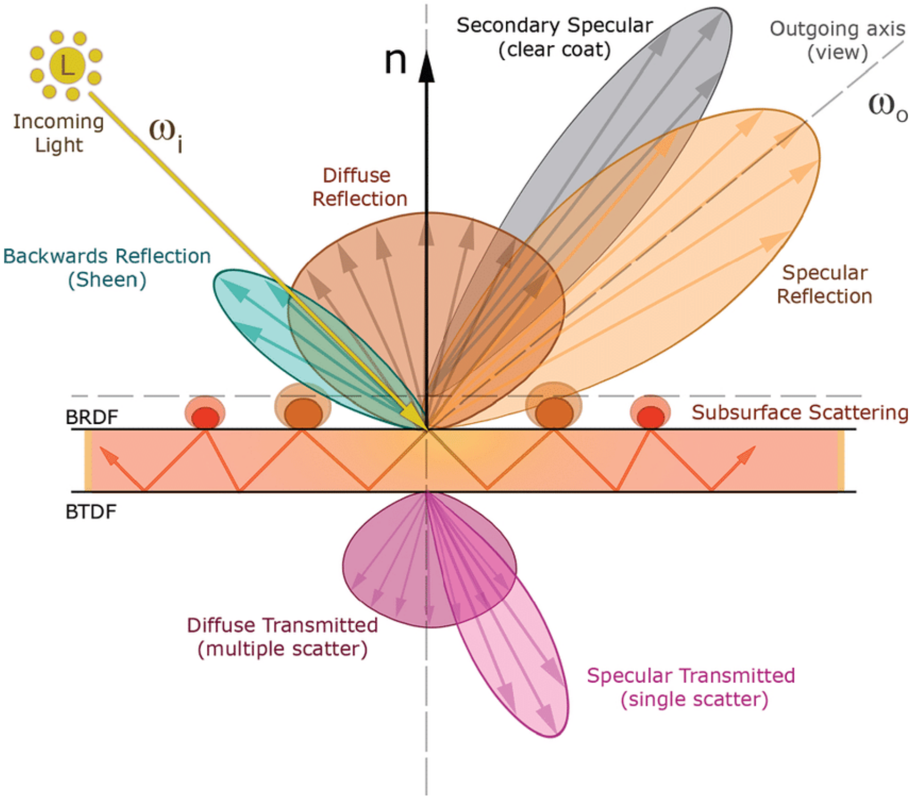
$$f_r(x, \omega_i, \omega_o) = w_1 \cdot f_{r1}(x, \omega_i, \omega_o) + w_2 \cdot f_{r2}(x, \omega_i, \omega_o)$$

# Principled BSDF

- Generalized BSDF model used in production (e.g. Blender)
  - Burley (Disney)
- Combines previous models
- Designed for artists
  - Not always physically plausible

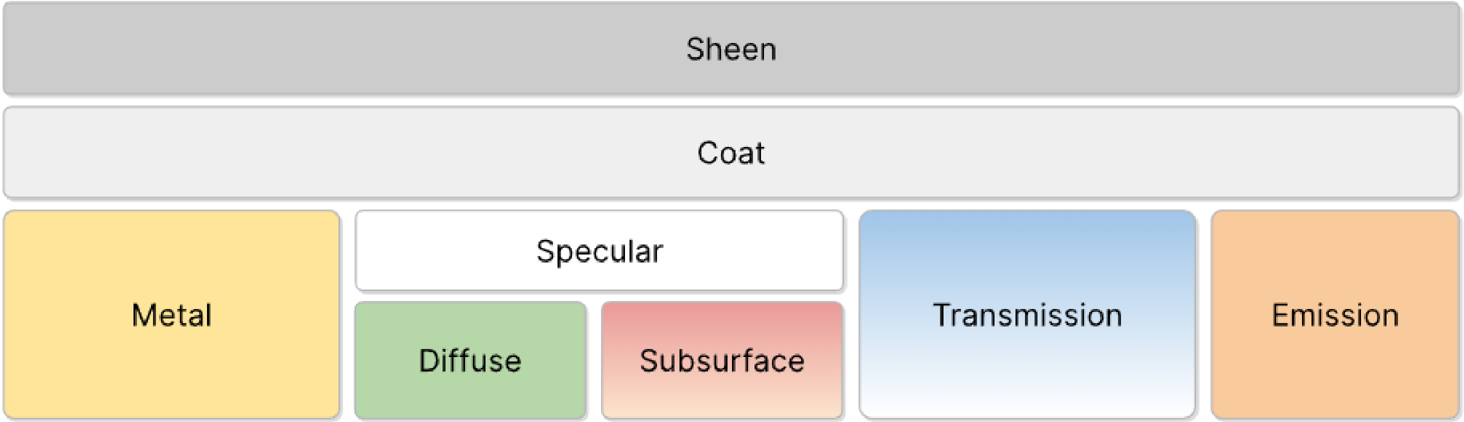
# Principled BSDF

- Mixture of multiple lobes



# Principled BSDF

- Mixture of multiple lobes




*Images from https://docs.blender.org/manual/en/latest/render/shader_nodes/shader/principled.html*

# Principled BSDF

- Diffuse
  - Modified version accounts for Fresnel
  - Matches real materials better

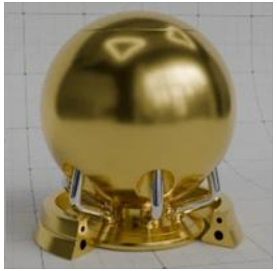
$$f_{diffuse} = \frac{baseColour}{\pi} F_D(\omega_i) F_D(\omega_o) |\omega_o \cdot n|$$

$$F_D(\omega) = 1 + (F_{D90} - 1)(1 - \omega_z)^5$$

$$F_{D90} = 0.5 + 2\,roughness\,|(H \cdot \omega_o)|^2$$

# Principled BSDF

- Metal
  - Cook-Torrance with GGX
  - Fresnel modelled as

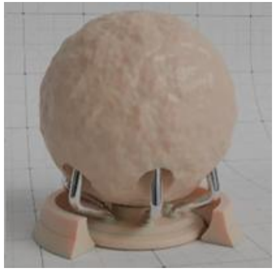


$$F_{metallic}(\omega) = baseColour + (1 - baseColour)(1 - (H \cdot \omega_o|)^5$$

- Specular (refraction)
  - Microfacet refraction for transmissive surfaces

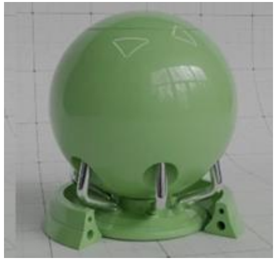
# Principled BSDF

- Subsurface
  - Can use physically correct volumetric path tracing or approximation
- Emissive
  - Emitted radiance
- Transmission
  - Weight for transmitting vs reflecting



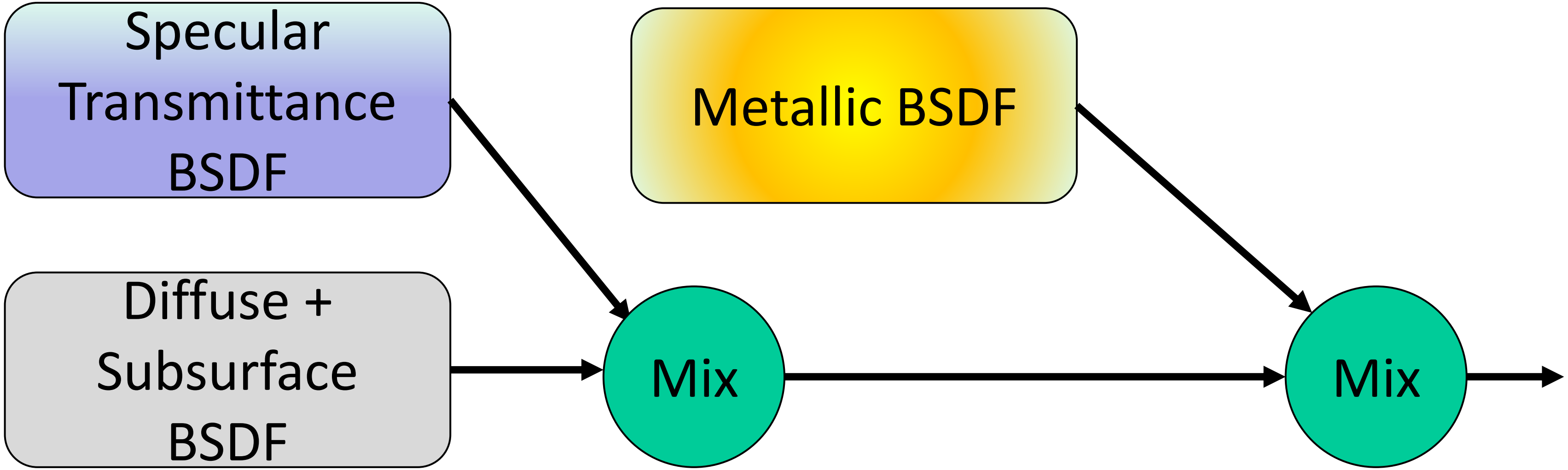
# Principled BSDF

- Clearcoat
  - One layer $\eta = 1.5$
  - Can include absorption and roughness
- Sheen
  - Models some fabrics



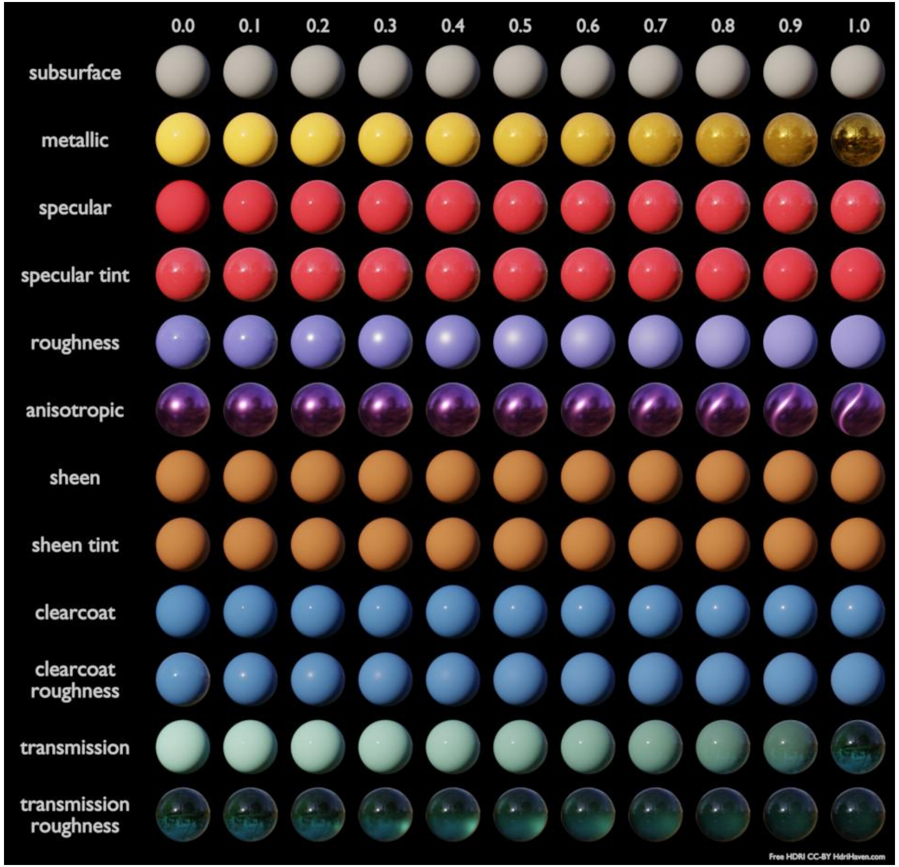
$$f_{sheen} = sheen \cdot ((1 - sheenTint) + sheenTint * tint)(1 - |H \cdot \omega_o|)^5$$

# Principled BSDF



# Principled BSDF

- Parameters

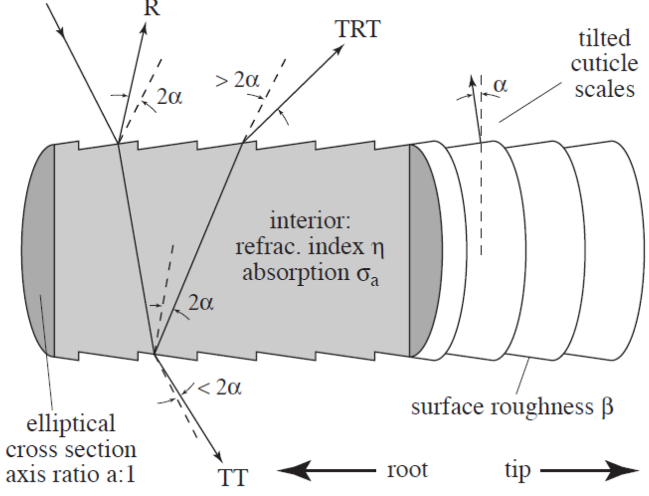


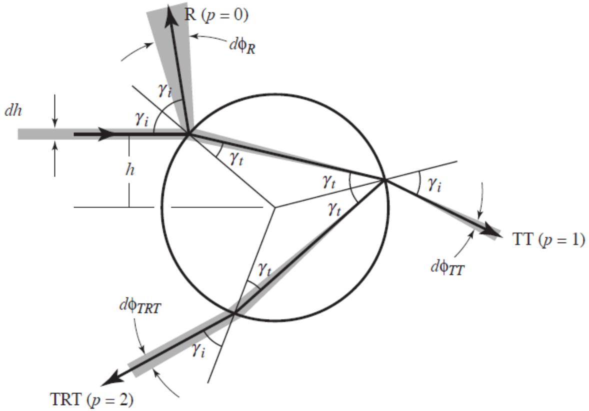
# Hair

- Specialized reflectance models for hair
- Accounts for hair geometry and scattering properties

# Hair Geometry

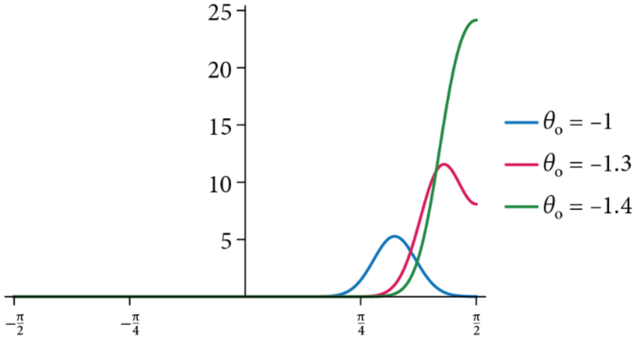
- Cuticle Scales
  - Both hair and fur
  - Fur includes central medulla with leading to more complicated scattering
- Absorption in cortex



# Hair

- Scattering Lobes
  - R, TT, TRT, TRRT
- Empirical formulation
  - Usually Wrapped Gaussian for azimuthal and 1D Gaussian for longitudinal (Marshner)
  - Can be importance sampled



# Hair

- Reflection controlled by roughness

# Hair

- Colour controlled by absorption



# Hair

- Importance of multiple scattering

# Games Workflow

- Specular vs Metalness workflow
- Different studios will use one or the other
- Metalness starting to dominate
  - Unity and Unreal
  - Unity can switch to specular

# Games Workflow

- Specular
  - Specular map to defines both the intensity and colour of reflections
  - Maps for diffuse and specular reflectance
  - Less principled but more control

# Games Workflow

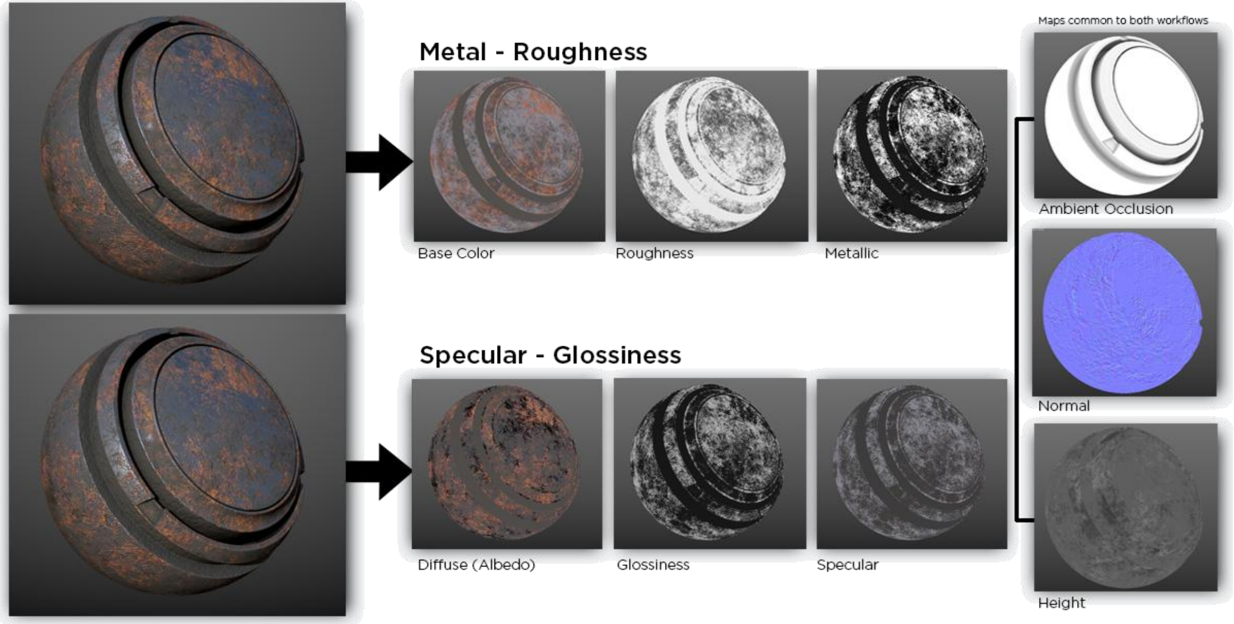
- Metalness
  - Uses a metalness map to specify if metallic
  - For metal, reflectance from base colour
  - For non metals, the reflectance is assumed to be low (e.g. 0.04), diffuse from diffuse map
  - More principled but less control

# Games Workflow

- Metalness
  - Uses a metalness map to specify if metallic
  - For metal, reflectance from base colour
  - For non metals, the reflectance is assumed to be low (e.g. 0.04), diffuse from diffuse map
  - More principled but less control

# Games Workflow

- Specular vs Metalness workflow

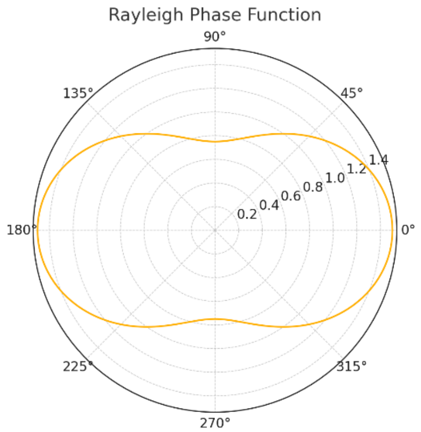


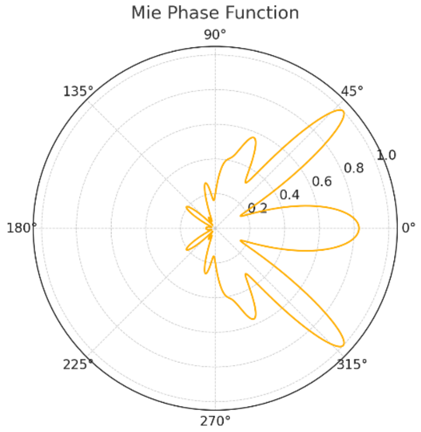
# Games Workflow

- Tools
  - Adobe Substance Painter
  - Marmoset Toolbag 4
  - Quixel Mixer

# Other Applications

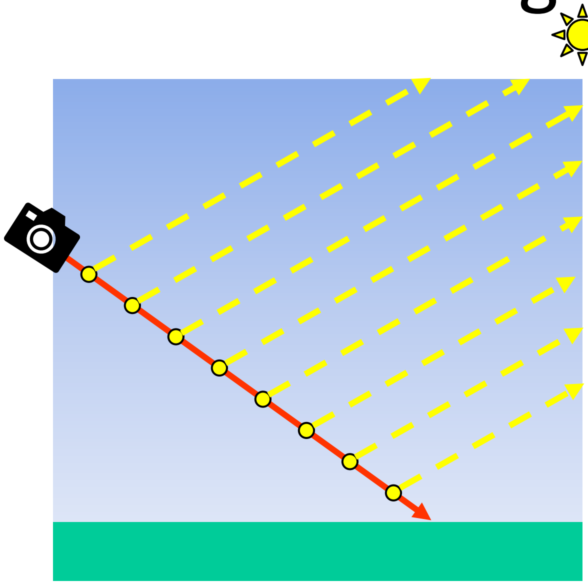
- Skies
  - – Simulates scattering in air
    - Rayleigh
  - – And particulates
    - Water, dust, pollen etc
    - Mie



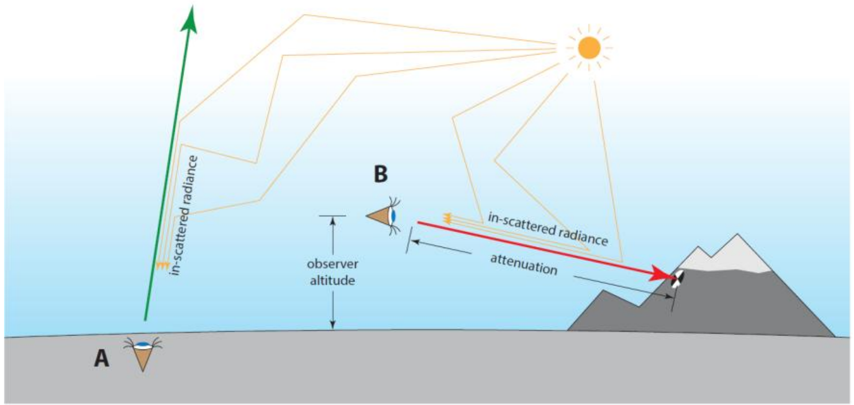
# Other Applications

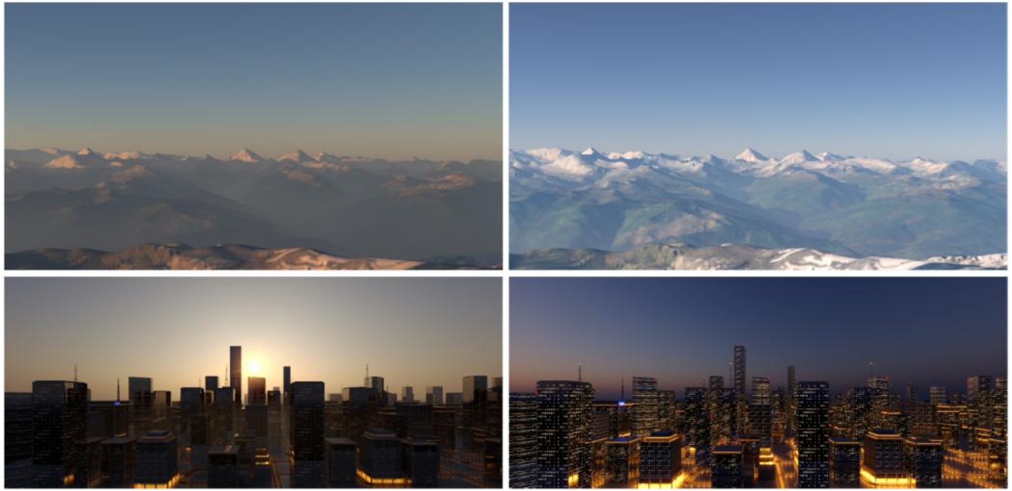
- Skies: Single scattering



# Other Applications

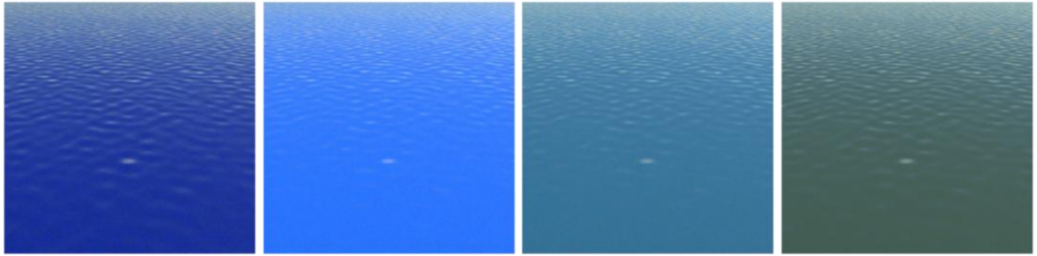
- Skies: Multiple Scattering



# Other Applications

- Water
  - Fresnel at surface
  - Subsurface scattering in water
  - Can use Mie scattering (mineral and algae)



Atlantic Mediterranean Baltic North Sea

# Machine Learning

- Has been used to compress materials
  - e.g. Real-Time Neural Appearance Models

# Machine Learning

- Has been used to compress materials
  - e.g. Skies
  - Generate clouds

# Materials

- Covered core PBR Materials
- Many more, e.g. diffractive models

# Questions?

## Computer Graphics

*Core Mathematical Concepts*

Dr. Tom Bashford-Rogers

WARWICK